

American International University-Bangladesh

AIUB Catalyst

Shamin Yasser

Kazi Shoaib Ahmed Saad

Faysal Ahammed Chowdhury

July 10, 2026

Contents

1 Setup 1
- 1.1 Sublime Build . . . 1
- 1.2 Template . . . 1
- 1.3 Input Gen . . . 1
- 1.4 Bash Script . . . 1

2 Number Theory 1
- 2.1 Euler Totient Function . . . 1
- 2.2 Phi 1 to N . . . 1
- 2.3 Segmented Sieve . . . 2
- 2.4 Extended GCD . . . 2
- 2.5 Linear Diophantine Equation . . . 2
- 2.6 Modular Inverse using EGCD . . . 2
- 2.7 Exclusion DP . . . 2
- 2.8 Legendres Formula . . . 2
- 2.9 Binary Expo . . . 2
- 2.10 Digit Sum of 1 to N . . . 2
- 2.11 Pollard Rho . . . 2
- 2.12 [Problem] How Many Bases - UVa 3
- 2.13 [Problem] Power Tower - CF . . . 3
- 2.14 Formula and Properties . . . 3
- 2.15 Mobius Formulas . . . 3

3 Combinatorics and Probability 3
- 3.1 Combinations . . . 3
- 3.2 nCr for any mod . . . 3
- 3.3 nCr without mod in O(r) . . . 3
- 3.4 Lucas Theorem . . . 3
- 3.5 Catalan Number . . . 3
- 3.6 Derangement . . . 3
- 3.7 Stars and Bars Theorem . . . 3
- 3.8 Properties of Pascal's Triangle . 3
- 3.9 Contribution Technique . . . 4
- 3.10 Probability and Expected Value . 4
- 3.11 Stirling Number of the First Kind 4
- 3.12 Stirling Number of the Second Kind . . . 4

4 Data Structure 4
- 4.1 Next Greater using Stack . . . 4
- 4.2 Segment Tree Lazy . . . 4
- 4.3 First Element Greater than K in a Range . . . 4
- 4.4 Number of Unique Element in a Range . . . 5
- 4.5 Max Subarray Sum in a Range . 5
- 4.6 Count of subarrays such that their XOR is 0 in a range with upd . . 5
- 4.7 Range Add, Set and Sum . . . 5
- 4.8 a[i] = a[i] * b + c in a Range . . 5
- 4.9 [Trick] Historic Information - GSS2 . . . 5
- 4.10 [Problem] Strongest Community - LOJ . . . 6
- 4.11 [Problem] Diablo - LOJ . . . 6
- 4.12 Merge Sort Tree with Point Upd 6
- 4.13 Sparse Table . . . 6
- 4.14 DSU . . . 6
- 4.15 MOs Algorithm . . . 7
- 4.16 MOs with Updates . . . 7
- 4.17 Wavelet Tree . . . 7
- 4.18 Persistent Segment Tree . . . 8
- 4.19 Trie for bit . . . 8
- 4.20 2D BIT . . . 8
- 4.21 Treap . . . 9
- 4.22 Centroid Decomposition . . . 9

5 Dynamic Programming 9
- 5.1 Knapsack 2 . . . 9
- 5.2 LIS using Segment Tree . . . 9
- 5.3 Digit DP . . . 9
- 5.4 Bitmask DP . . . 10
- 5.5 MCM DP . . . 10
- 5.6 Number of Unique Subsequence . 10
- 5.7 [Problem] Rose Land - SUST 24 10
- 5.8 DP with DS . . . 11
- 5.9 [Problem] Sub-Palindromic Tree 11
- 5.10 SOS DP . . . 11

6 Graph Theory 11
- 6.1 Binary Lifting and LCA . . . 11
- 6.2 LCA and Sparse Table on Tree . 11
- 6.3 Dijkstra . . . 12
- 6.4 Bellman Ford . . . 12
- 6.5 Floyd Warshall . . . 12
- 6.6 Strongly Connected Components 12
- 6.7 Articulation Points . . . 12
- 6.8 Find Bridges . . . 12
- 6.9 DSU on Tree . . . 12
- 6.10 Heavy-Light Decomposition . . . 13
- 6.11 Inverse Graph . . . 13
- 6.12 [Problem] Rooted Tree - Hackerrank . . . 13

7 String 13
- 7.1 Hashing . . . 13
- 7.2 Hashing with Updates . . . 14
- 7.3 Hashing with Upd and Deletes . 14
- 7.4 Hashing on Tree . . . 15
- 7.5 Compare 2 strings Lexicographically . . . 15
- 7.6 KMP . . . 15
- 7.7 KMP Automata . . . 15
- 7.8 Prefix Occurance Count . . . 15
- 7.9 Number of palindormic substring in L to R using Wavelet Tree . . 16
- 7.10 Trie . . . 16
- 7.11 Manachar . . . 16
- 7.12 Suffix Array . . . 16

8 Game Theory 17
- 8.1 Notes . . . 17

9 Flow 17
- 9.1 Flow with L H bound . . . 17
- 9.2 Hopcroft Karp . . . 18
- 9.3 Max-Flow . . . 18
- 9.4 Max Flow with Min Cost . . . 19
- 9.5 Flow Properties . . . 19

10 Misc. 19
- 10.1 [Trick] Median of Medians . . . 19
- 10.2 Ternary Search . . . 19
- 10.3 Pigeonhole Principle . . . 20
- 10.4 Matrix Expo . . . 20
- 10.5 2D Prefix Sum . . . 20
- 10.6 2D Static Range Update . . . 20
- 10.7 CHECK SET CLR . . . 20
- 10.8 Graph Directions . . . 20
- 10.9 Custom Hash . . . 20
- 10.10 Coordinate Compression . . . 20
- 10.11 Submask Generator . . . 20
- 10.12 Custom Comparators . . . 20
- 10.13 MOD Operations . . . 20
- 10.14 Mex with Array Updates . . . 20
- 10.15 Check Eq AlphaQ . . . 20

11 Geometry 21
- 11.1 Convex Hull . . . 21
- 11.2 Area of 2 polygon intersection . . 21
- 11.3 Point Segment Distance . . . 21
- 11.4 Three Point Circle . . . 21
- 11.5 Circle Line Intersection . . . 21

12 Solid Geometry 23
- 12.1 Rectangular Solid . . . 23
- 12.2 Cube . . . 23
- 12.3 Cylinder . . . 23
- 12.4 Prism . . . 24
- 12.5 Pyramid . . . 24
- 12.6 Right Circular Cone . . . 24
- 12.7 Sphere . . . 24

1 Setup

1.1 Sublime Build

```
"shell_cmd": "g++ -std=c++17 -o
    \"$file_base_name\" \"$file\" &&
    timeout 2.5s ./\"$file_base_name\" <
    input.txt > output.txt",
"file_regex":
    "^(..[^:]*):([0-9]+):?([0-9]+)?:?
    (.*)$",
"working_dir": "${file_path}",
"selector": "source.c, source.c++"
```

1.2 Template

```
#include<bits/stdc++.h>
#include<ext/pb_ds/assoc_container.hpp>
#include<ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
using namespace std;
template <typename T> using o_set =
    tree<T, null_type, less<T>,
    rb_tree_tag,
    tree_order_statistics_node_update>;
typedef long long ll;
int32_t main() {
  ios_base::sync_with_stdio(0);
  cin.tie(0);
}
```

1.3 Input Gen

```
mt19937_64 rnd(chrono::steady_clock::now
    ().time_since_epoch().count());
ll get_rand(ll l, ll r) {
  assert(l <= r);
  return l + rnd() % (r - l + 1);
}
```

1.4 Bash Script

```
// run -> bash script.sh

set -e
g++ code.cpp -o code
g++ gen.cpp -o gen
g++ brute.cpp -o brute
for((i = 1; ; ++i)); do
    ./gen $i > input_file
    ./code < input_file > myAnswer
    ./brute < input_file > correctAnswer
    diff -Z myAnswer correctAnswer >
        /dev/null || break
    echo "Passed test: "  $i
done
echo "WA on the following test:"
cat input_file
echo "Your answer is:"
cat myAnswer
echo "Correct answer is:"
cat correctAnswer
```

2 Number Theory

2.1 Euler Totient Function

```
// Time: O(√N)
map<int, int> dp; // memo
int phi(int n) {
  if (dp.count(n)) return dp[n];
  int ans = n, m = n;
  for (int i = 2; i * i <= m; i++) {
    if (m % i == 0) {
      while (m % i == 0) m /= i;
      ans = ans / i * (i - 1);
    }
  }
  if (m > 1) ans = ans / m * (m - 1);
  return dp[n] = ans;
}
```

2.2 Phi 1 to N

```
void phi_1_to_n(int n) {
  vector<int> phi(n + 1);
  for (int i = 0; i <= n; i++)
    phi[i] = i;
  for (int i = 2; i <= n; i++) {
```

```
    if (phi[i] == i) {
      for (int j = i; j <= n; j += i)
        phi[j] -= phi[j] / i;
    }
  }
}
```

2.3 Segmented Sieve

```
vector<char> segmentedSieve(ll L, ll R) {
  // generate all primes up to sqrt(R)
  ll lim = sqrt(R);
  vector<char> mark(lim + 1, false);
  vector<ll> primes;
  for (ll i = 2; i <= lim; ++i) {
    if (!mark[i]) {
      primes.emplace_back(i);
      for (ll j = i * i; j <= lim; j +=
          i) mark[j] = true;
    }
  }
  vector<char> isPrime(R - L + 1, true);
  for (ll i : primes)
    for (ll j = max(i * i, (L + i - 1) /
        i * i); j <= R; j += i)
      isPrime[j - L] = false;
  if (L == 1) isPrime[0] = false;
  return isPrime;
}
```

2.4 Extended GCD

```
// ax + by = gcd(a,b)
int egcd(int a, int b, int& x, int& y) {
  if (b == 0) {
    x = 1, y = 0;
    return a;
  }
  int x1, y1;
  int d = egcd(b, a % b, x1, y1);
  x = y1;
  y = x1 - y1 * (a / b);
  return d;
}
```

2.5 Linear Diophantine Equation

```
// ax + by = c, find any x and y
bool find_any_solution(int a, int b, int
    c, int &x0, int &y0, int &g) {
  g = egcd(abs(a), abs(b), x0, y0);
  if (c % g) return false;
  x0 *= c / g;
  y0 *= c / g;
  if (a < 0) x0 = -x0;
  if (b < 0) y0 = -y0;
  return true;
}
void shift_solution(int & x, int & y,
    int a, int b, int cnt) {
  x += cnt * b;
  y -= cnt * a;
}
int find_all_solutions(int a, int b, int
    c, int minx, int maxx, int miny, int
    maxy) {
  int x, y, g;
  if (!find_any_solution(a, b, c, x, y,
      g)) return 0;
  a /= g, b /= g;
  int sign_a = a > 0 ? +1 : -1;
  int sign_b = b > 0 ? +1 : -1;
  shift_solution(x, y, a, b, (minx - x)
      / b);
  if (x < minx) shift_solution(x, y, a,
      b, sign_b);
  if (x > maxx) return 0;
  int lx1 = x;
  shift_solution(x, y, a, b, (maxx - x)
      / b);
  if (x > maxx) shift_solution(x, y, a,
      b, -sign_b);
  int rx1 = x;
  shift_solution(x, y, a, b, -(miny - y)
      / a);
  if (y < miny) shift_solution(x, y, a,
      b, -sign_a);
  if (y > maxy) return 0;
  int lx2 = x;
  shift_solution(x, y, a, b, -(maxy - y)
      / a);
  if (y > maxy) shift_solution(x, y, a,
      b, sign_a);
  int rx2 = x;
  if (lx2 > rx2) swap(lx2, rx2);
  int lx = max(lx1, lx2);
  int rx = min(rx1, rx2);
  if (lx > rx) return 0;
  return (rx - lx) / abs(b) + 1;
}
```

2.6 Modular Inverse using EGCD

```
// finding inverse(a) modulo m
int x, y;
int g = extended_euclidean(a, m, x, y);
if (g != 1) cout << "No solution!";
else {
  x = (x % m + m) % m;
  cout << x << endl;
}
```

2.7 Exclusion DP

```
ll f[N], g[N];
for (int i = N - 1; i >= 1; i--) {
  f[i] = nC4(div_cnt[i]);
  g[i] = f[i];
  for (int j = i + i; j < N; j += i) {
    g[i] -= g[j];
  }
}
```

- Here, $f[i]$ = how many pairs/k-tuple such that their gcd is i or it's multiple (count of pairs those are divisible by i).
- $g[i]$ = how many pairs/k-tuple such that their gcd is i.
- $g[i] = f[i] - \sum_{i|j} g[j]$.
- **Sum of all pair gcd:**

  We know, how many pairs are there such that their gcd is i for every i (1 to n). So now, $\sum_{i=1}^{n} g[i] * i$.
- **Sum of all pair lcm** $(1 <= i, j <= n)$**:** We know, $\text{lcm}(a,b) = \frac{a*b}{\text{gcd}(a,b)}$.
- Now, $f[i]$ = All pair product sum of those, whose gcd is i or it's multiple.
- $g[i]$ = All pair product sum of those, whose gcd is i.
- Ans $= \sum_{i=1}^{n} \frac{g[i]}{i}$.
- All pair product sum $= (a_1 + a_2 + \cdots + a_n) * (a_1 + a_2 + \cdots + a_n)$

2.8 Legendres Formula

```
// n!/p^x  - you will get the largest x
int legendre(int n, int p) {
  int ex = 0;
  while(n) {
    ex += (n / p);
    n /= p;
  }
  return ex;
}
```

2.9 Binary Expo

```
int power(int x, long long n, int mod) {
  int ans = 1 % mod;
  while (n > 0) {
    if (n & 1) {
      ans = 1LL * ans * x % mod;
    }
    x = 1LL * x * x % mod;
    n >>= 1;
  }
  return ans;
}
```

2.10 Digit Sum of 1 to N

```
// for n=10, ans = 1+2+...+9+1+0
ll solve(ll n) {
  ll res = 0, p = 1;
  while (n / p > 0) {
    ll left = n / (p * 10);
    ll cur  = (n / p) % 10, right = n % p;
    res += left * 45 * p;
    res += (cur * (cur - 1) / 2) * p;
    res += cur * (right + 1); p *= 10;
  } return res;
}
int totalDigits(int n) {
  if ( n <= 0 ) return 0;
  int p = 1, total = 0;
  for ( int d = 1; d <= 18; d++ ) {
    int L = p, R = min(n, p * 10 - 1);
    if ( R >= L ) total += (R - L + 1) *
        1LL * d;
    p *= 10; if ( p > n ) break;
  } return total;
}
```

2.11 Pollard Rho

```
namespace PollardRho {
mt19937 rnd(chrono::steady_clock::now().
    time_since_epoch().count());
const int P = 1e6 + 9;
ll seq[P];
int primes[P], spf[P];
inline ll add_mod(ll x, ll y, ll m) {
  return (x += y) < m ? x : x - m;
}
inline ll mul_mod(ll x, ll y, ll m) {
  ll res = __int128(x) * y % m;
  return res;
  // ll res = x * y - (ll)((long double)x
      * y / m + 0.5) * m;
  // return res < 0 ? res + m : res;
}
inline ll pow_mod(ll x, ll n, ll m) {
  ll res = 1 % m;
  for (; n; n >>= 1) {
    if (n & 1) res = mul_mod(res, x, m);
    x = mul_mod(x, x, m);
  }
  return res;
}
  // O(it * (logn)^3), it = number of rounds
    performed
inline bool miller_rabin(ll n) {
  if (n <= 2 || (n & 1 ^ 1)) return (n
      == 2);
  if (n < P) return spf[n] == n;
  ll c, d, s = 0, r = n - 1;
  for (; !(r & 1); r >>= 1, s++) {}
  // each iteration is a round
  for (int i = 0; primes[i] < n &&
      primes[i] < 32; i++) {
    c = pow_mod(primes[i], r, n);
    for (int j = 0; j < s; j++) {
      d = mul_mod(c, c, n);
      if (d == 1 && c != 1 && c != (n -
          1)) return false;
      c = d;
    }
    if (c != 1) return false;
  }
  return true;
}
void init() {
  int cnt = 0;
  for (int i = 2; i < P; i++) {
    if (!spf[i]) primes[cnt++] = spf[i]
        = i;
    for (int j = 0, k; (k = i *
        primes[j]) < P; j++) {
      spf[k] = primes[j];
      if (spf[i] == spf[k]) break;
    }
  }
}
// returns O(n^(1/4))
ll pollard_rho(ll n) {
  while (1) {
    ll x = rnd() % n, y = x, c = rnd() %
        n, u = 1, v, t = 0;
    ll *px = seq, *py = seq;
    while (1) {
      *py++ = y = add_mod(mul_mod(y, y,
          n), c, n);
      *py++ = y = add_mod(mul_mod(y, y,
          n), c, n);
      if ((x = *px++) == y) break;
      v = u;
```

```
      u = mul_mod(u, abs(y - x), n);
      if (!u) return __gcd(v, n);
      if (++t == 32) {
        t = 0;
        if ((u = __gcd(u, n)) > 1 && u <
            n) return u;
      }
    }
    if (t && (u = __gcd(u, n)) > 1 && u
        < n) return u;
  }
}
vector<ll> factorize(ll n) {
  if (n == 1) return vector <ll>();
  if (miller_rabin(n)) return vector<ll>
      {n};
  vector <ll> v, w;
  while (n > 1 && n < P) {
    v.push_back(spf[n]);
    n /= spf[n];
  }
  if (n >= P) {
    ll x = pollard_rho(n);
    v = factorize(x);
    w = factorize(n / x);
    v.insert(v.end(), w.begin(), w.end());
  }
  return v;
}
}
```

2.12 [Problem] How Many Bases - UVa

```
  // Given a number N^M, find out the
     number of integer bases in which it
     has exactly T trailing zeroes.
int solve_greater_or_equal(vector<int>
    e, int t) {
  int ans = 1;
  for (auto i : e) {
    ans = 1LL * ans * (i / t + 1) % mod;
  }
  return ans;
}
// e contains e1, e2 -> p1^e1, p2^e2
int solve_equal(vector<int> e, int t) {
  return (solve_greater_or_equal(e, t) -
      solve_greater_or_equal(e, t + 1) +
      mod) % mod;
}
```

2.13 [Problem] Power Tower - CF

```
    // A sequence w1, w2, ..., wn and Q
    queries, l and r will be given.
    Calculate w_l^(w_{l+1}^(...^(w_r)))
// n^x mod m = n^(phi(m)+x mod phi(m)) mod m
inline int MOD(int x, int m) {
  if (x < m) return x;
  return x % m + m;
}
int power(int n, int k, int mod) {
  int ans = MOD(1, mod);
  while (k) {
    if (k & 1) ans = MOD(ans * n, mod);
    n = MOD(n * n, mod);
    k >>= 1;
  }
  return ans;
}
int f(int l, int r, int m) {
  if (l == r) return MOD(a[l], m);
  if (m == 1) return 1;
  return power(a[l], f(l + 1, r,
      phi(m)), m);
}
```

2.14 Formula and Properties

- $\phi(n) = n \cdot \frac{p_1-1}{p_1} \cdot \frac{p_2-1}{p_2} \cdots$
- $\phi(p^e) = p^e - \frac{p^e}{p} = p^e \cdot \frac{p-1}{p}$
- For $n > 2$, $\phi(n)$ is always even.
- $\sum_{d|n} \phi(d) = n$
- NOD: $(e_1 + 1) \cdot (e_2 + 1) \cdots$
- SOD: $\frac{p_1^{e_1+1}-1}{p_1-1} \cdot \frac{p_2^{e_2+1}-1}{p_2-1} \cdots$
- **Digit Count of n:** $\lfloor \log_{10}(n) \rfloor + 1$
- **Arithmetic Progression Sum:** $\frac{n}{2} \cdot (a+p)$, $\frac{n}{2} \cdot (2a + (n-1)d)$
- **Geometric Sum:** $S_n = a \cdot \frac{r^n-1}{r-1}$
- $(1^2 + 2^2 + \cdots + n^2) = \frac{n(n+1)(2n+1)}{6}$
- $(1^3 + 2^3 + \cdots + n^3) = \frac{n^2(n+1)^2}{4}$
- $(2^2 + 4^2 + \cdots + (2n)^2) = \frac{2n(n+1)(2n+1)}{3}$
- $(1^2 + 3^2 + \cdots + (2n-1)^2) = \frac{n(2n-1)(2n+1)}{3}$
- $(2^3 + 4^3 + \cdots + (2n)^3) = 2n^2(n+1)^2$
- $(1^3 + 3^3 + \cdots + (2n-1)^3) = n^2(2n^2-1)$
- For any number n and bases $> \sqrt{n}$, there will be no representation where the number contains 0 at its second least significant digit. So it is enough to check for bases $\leq \sqrt{n}$.
- For some x and y, let's try to find all m such that $x \bmod m \equiv y \bmod m$. We can rearrange the equation into $(x-y) \equiv 0 \pmod m$. Thus, if m is a factor of $|x-y|$, then x and y will be equal modulo m.

2.15 Mobius Formulas

```
void mobius() { mob[1] = 1;
  for (int i = 2; i < N; i++) { mob[i]--;
    for (int j = i + i; j < N; j += i)
      mob[j] -= mob[i];
  }
}
```

- **Coprime Pairs** $(\sum_{i=1}^{n}\sum_{j=1}^{n}[\gcd(i,j) = 1])$: $f(n) = \sum_{d=1}^{n} \mu(d) \left\lfloor \frac{n}{d} \right\rfloor^2$
- **Sum of GCD** $(\sum_{i=1}^{n}\sum_{j=1}^{n} \gcd(i,j))$: $f(n) = \sum_{l=1}^{n} \left\lfloor \frac{n}{l} \right\rfloor^2 \sum_{k|l} k\mu\left(\frac{l}{k}\right)$
- **Sum of LCM** $(\sum_{i=1}^{n}\sum_{j=1}^{n} \mathrm{lcm}(i,j))$: $f(n) = \sum_{l=1}^{n} \left(\frac{(1+\lfloor n/l \rfloor)\lfloor n/l \rfloor}{2}\right)^2 g(l)$ (where $g(p^k) = p^k - p^{k+1}$)
- **Sum of ALL k-tuples GCD:** $\sum f(i) \times \phi(i)$
- $g(l) = \sum_{d|l} \mu(d) ld$

3 Combinatorics and Probability

3.1 Combinations

```
// Prime Mod in O(n)
void prec() {
  fact[0] = 1;
  for (int i = 1; i < N; i++) {
    fact[i] = 1ll * fact[i - 1] * i % mod;
  }
  ifact[N - 1] = inverse(fact[N - 1]);
  for (int i = N - 2; i >= 0; i--) {
    ifact[i] = 1ll * ifact[i + 1] * (i +
        1) % mod;
  }
}
int nCr(int n, int r) {
  if (r > n) return 0;
  return 1ll * fact[n] * ifact[r] % mod
      * ifact[n - r] % mod;
}
int nPr(int n, int r) {
  if (r > n) return 0;
  return 1ll * fact[n] * ifact[n - r] %
      mod;
}
```

3.2 nCr for any mod

```
// Time: O(n^2)
// nCr = (n-1)C(r-1) + (n-1)Cr
for (int i = 0; i < N; i++) {
  C[i][i] = 1;
  for (int j = 0; j < i; j++) {
    C[i][j] = (C[i - 1][j] + C[i - 1][j
        - 1]) % mod;
  }
}
```

3.3 nCr without mod in O(r)

```
ll nCk(ll n, ll k) {
  double res = 1;
  for (ll i = 1; i <= k; ++i)
    res = res * (n - k + i) / i;
  return (ll)(res + 0.01);
}
```

3.4 Lucas Theorem

```
// returns nCr modulo mod where mod is a
    prime
// Complexity: ?
ll Lucas(ll n, ll r) {
  if (r < 0 || r > n) return 0;
  if (r == 0 || r == n) return 1;
  if (n >= MOD) {
    return (Lucas(n / MOD, r / MOD) %
        MOD * Lucas(n % MOD, r % MOD) %
        MOD) % MOD;
  }
  return (((fact[n] * invFact[r]) % MOD)
      * invFact[n - r]) % MOD;
}
```

3.5 Catalan Number

```
const int MOD = 1e9 + 7, int MAX = 1e7;
int catalan[MAX];
void init(ll n) {
  catalan[0] = catalan[1] = 1;
  for ( ll i = 2; i <= n; i++ ) {
    catalan[i] = 0;
    for ( ll j = 0; j < i; j++ ) {
      catalan[i] += ( catalan[j] *
          catalan[i - j - 1] ) % MOD;
      if ( catalan[i] >= MOD ) {
        catalan[i] -= MOD;
      }
    }
  }
}
```

3.6 Derangement

```
    // number of combinations such that
    a_i! = i of a permuation a
const int N = 1e6 + 100, int p = 1e9 + 7;
ll der[N];
void countDer() {
  der[1] = 0; der[2] = 1;
  for (ll i = 3; i <= N; ++i) {
    der[i] = (i - 1) % p * (der[i - 1] %
        p + der[i - 2] % p);
    der[i] %= p;
  }
}
```

3.7 Stars and Bars Theorem

- Find the number of k-tuples of non-negative integers whose sum is n. $\binom{n+k-1}{n}$
- Find the number of k-tuples of non-negative integers whose sum is $\leq n$. $\binom{n+k}{k}$
- Combination with Repetition (choose k elements from n objects, same element can be chosen multiple times). $\binom{n+k-1}{k}$
- How many ways to go from $(0,0)$ to (n,m). $\binom{n+m}{m}$

3.8 Properties of Pascal's Triangle

- $(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^k b^{n-k}$
- $(k+1)^n = \sum_{i=0}^{n} k^i \cdot \binom{n}{i}$
- $\sum_{i=0}^{n} \binom{n}{i} = 2^n$
- $\binom{n}{k} = \frac{n}{k}\binom{n-1}{k-1}$
- $\sum_{k=0}^{m} \binom{n+k}{k} = \binom{n+m+1}{m}$
- $\binom{n}{0}^2 + \binom{n}{1}^2 + \cdots + \binom{n}{n}^2 = \binom{2n}{n}$
- $1\binom{n}{1} + 2\binom{n}{2} + \cdots + n\binom{n}{n} = n\,2^{n-1}$

3.9 Contribution Technique

- Sum of all pair sums: $\sum_{i=1}^{n}\sum_{j=1}^{n}(a_i + a_j)$ Every element will be added $2n$ times. $\sum_{i=1}^{n}(2 \times n \times a_i) = 2 \times n \times \sum_{i=1}^{n} a_i$.
- Sum of all subarray sums — $\sum_{i=1}^{n}(a_i \times i \times (n-i+1))$.
- Sum of all Subsets sums — $\sum_{i=1}^{n}\left(2^{n-1} \times a_i\right)$.
- Product of all pair product — $\prod_{i=1}^{n}\left(a_i^{2\times n}\right)$.
- XOR of subarray XORS — How many subarrays does an element have? $(i \cdot (n-i+1)$ times). If subarray count is odd then this element can contribute in total XORs.
- Sum of max minus min over all subset — Sort the array. $Min = 2^{n-i}$, $Max = 2^{i-1}$. $\sum_{i=1}^{n}\left(a_i \cdot 2^{i-1} - a_i \cdot 2^{n-i}\right)$
- Sum using bits — $\sum_{k=0}^{30}\left(cnt_k[1] \times 2^k\right)$.
- Sum of Pair XORs — XOR will 1 if two bits are different $\sum_{k=0}^{30}\left(cnt_k[0] \times cnt_k[1] \times 2^k\right)$.
- Sum of Pair ANDs — $\sum_{k=0}^{30}\left(cnt_k[1]^2 \times 2^k\right)$.
- Sum of Pair ORs — $\sum_{k=0}^{30}\left(\left(cnt_k[1]^2 + 2 \times cnt_k[1] \times cnt_k[0]\right) \times 2^k\right)$.
- Sum of Subset XORs — where $cnt0! = 0$ $\sum_{k=0}^{30}\left(2^{cnt_k[1]+cnt_k[0]-1} \times 2^k\right)$.
- Sum of Subset ANDs — $\sum_{k=0}^{30}\left(\left(2^{cnt_k[1]} - 1\right) \times 2^k\right)$.
- Sum of Subset ORs — $\sum_{k=0}^{30}\left(\left(2^n - 2^{cnt_k[0]}\right) \times 2^k\right)$.
- Sum of subarray XORs — Convert to prefix xor, then solve for pairs.
- Sum of product of all subsequence — $\prod_{i=1}^{n}(a_i + 1) - 1$. Example array — $[a, b]$ the subsequences are $\{a\}, \{b\}, \{a, b\}$ so ans is $a + b + (a \cdot b)$

3.10 Probability and Expected Value

- Expected Value: $E = \frac{\text{Sum of all possible values}}{\text{Total number of outcomes}}$
- Expected Value: $E = \sum_{i=1}^{n} P_i \cdot i$
- Variance: $V(x) = E(x^2) - \{E(x)\}^2$
- Expected Value with DP: $E(i) = \sum P(i \to j) \times (R(i \to j) + E(j))$ Where $R()$ is Immediate Reward(cost/count)
- Linearity of Expectation:
  - $E[X + Y] = E[X] + E[Y]$
  - $E[\text{Total}] = E[I_1] + E[I_2] + \cdots = \sum P(I_i = 1)$

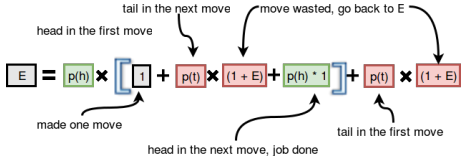
  We need to define the Indicator Random Variable (I) correctly. Some Examples below —

  Problem-1: Find E[correct hats] in a random permutation. Indicator I_i: "Does person i get their own hat?"

  Problem-2: Find E[total inversions] in a random permutation. Indicator I_{ij}: "Is pair (i, j) an inversion?"

  Problem-3: Given a string S, delete a random index until it becomes empty. Find the expected count of palindromes seen (exclude S, include empty string). Indicator I_L: "Is the string of length L a palindrome?"

  Problem-4: Pick N random integers from $[1, k]$. Merge consecutive equal values (e.g., $1, 1, 2, 2, 1, 1 \to 1, 2, 1$). Find the expected final length. Indicator I_i: "Is item i different from item $i-1$?"
- To get two consecutive heads, what is the expected number of tosses?



- To get n heads, what is the expected number of tosses? Let's define: to get n heads, we need to toss $E(n)$ times. Now — I can get a head; I need to toss $E(n-1)$ more times, or if I get a tail; I need to toss $E(n)$ times. So, the recurrence is: $E(n) = 0.5 \cdot (1 + E(n-1)) + 0.5 \cdot (1 + E(n))$
- You have n bulbs, all of which are initially off. In each move, you randomly select one bulb. If the selected bulb is **off**, you toss a coin:
  - If you get head, you turn it on.
  - If you get tail, you do nothing.

  If the bulb is already **on**, you skip that move (nothing happens).

  Now, what is the expected number of moves required to turn all bulbs on?

  The coin is not fair — the probability of getting tail is p. This problem can also be solved recursively.

  Let's assume at some moment, x bulbs are already on, and the expected number of moves needed from here is $e(x)$.

  The probability of picking an already on bulb is $\frac{x}{n}$. In that case, the expected number of moves is $\frac{\frac{x}{n}}{\frac{x}{n}} \times (1 + e(x))$.

  The probability of picking an off bulb is $\frac{n-x}{n}$.

  Now two things can happen:
  - With probability p, you get tail, so you stay at the same state ($e(x)$ more moves).
  - With probability $(1-p)$, you get head, so one more bulb turns on ($e(x+1)$ moves from there).

  So, the recurrence relation is:

$$e(x) = \frac{x}{n}(1 + e(x)) + \frac{n-x}{n}(p(1 + e(x)) + (1-p)(1 + e(x+1)))$$

3.11 Stirling Number of the First Kind

- Count permutation according to their number of cycles.
- $S(n, k)$ counts the number of permutations of n elements with k disjoint cycles.
- $S(n, k) = (n-1)S(n-1, k) + S(n-1, k-1)$, $S(0, 0) = 1$, $S(n, 0) = S(0, n) = 0$
- $S(n, 1) = (n-1)!$
- $S(n, n-1) = \binom{n}{2}$
- $\sum_{k=0}^{n} S(n, k) = n!$

3.12 Stirling Number of the Second Kind

- Number of ways to partition a set of n objects into k non-empty subsets.
- $S(n, k) = k\,S(n-1, k) + S(n-1, k-1)$, $S(0, 0) = 1$, $S(n, 0) = S(0, n) = 0$
- $S(n, 2) = 2^{n-1} - 1$
- $S(n, k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$
- $S(n, k) \cdot k!$ = number of ways to color n nodes using colors from 1 to k such that each color is used at least once.

4 Data Structure

4.1 Next Greater using Stack

```
stack<pair<int, int>> st;
  int right[n + 1];     // a[i] is maximum
   from index i to right[i]
for (int i = 1; i <= n; i++) {
  while (!st.empty() and st.top().first
      < a[i]) {
    right[st.top().second] = i - 1;
    st.pop();
  }
  st.push({a[i], i});
}
while (!st.empty()) {
  right[st.top().second] = n;
  st.pop();
}
```

4.2 Segment Tree Lazy

```
struct ST {
  int tree[4 * N], lazy[4 * N];
  void push(int n, int b, int e) {
    if(lazy[n] == 0) return;
    tree[n] += lazy[n] * (e - b + 1); //
        change here
    if(b != e) {
      int l = n << 1, r = l + 1;
      lazy[l] += lazy[n]; // change here
      lazy[r] += lazy[n]; // change here
    }
    lazy[n] = 0;
  }
  void build(int n, int  b, int e) {
    lazy[n] = 0;
    if(b == e) {
      tree[n] = a[b]; // change here
      return;
    }
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    build(l, b, mid);
    build(r, mid + 1, e);
    tree[n] = tree[l] + tree[r]; //
        change here
  }
  void upd(int n, int b, int e, int i,
      int j, int x) {
    push(n, b, e);
    if(b > j || e < i) return;
    if(b >= i && e <= j) {
      lazy[n] += x; // change here
      push(n, b, e);
      return;
    }
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    upd(l, b, mid, i, j, x);
    upd(r, mid + 1, e, i, j, x);
    tree[n] = tree[l] + tree[r]; //
        change here
  }
  int query(int n, int b, int e, int i,
      int j) {
    push(n, b, e);
    if(b > j || e < i) return 0; //
        return appropriate value
    if(b >= i && e <= j) return tree[n];
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    int L = query(l, b, mid, i, j);
    int R = query(r, mid + 1, e, i, j);
    return L + R; // change here
  }
} st;
```

4.3 First Element Greater than K in a Range

```
// tree[n] = max in range
int get_first(int n, int b, int e, int
    i, int j, int x) { // return index
  push(n, b, e);
  if (b > j || e < i) return -1;
  if (tree[n] <= x) return -1;
  if (b == e) return b;
  int mid = (b + e) >> 1, l = n << 1, r
      = l + 1;
  int left = get_first(l, b, mid, i, j,
      x);
  if (left != -1) return left;
  return get_first(r, mid + 1, e, i , j,
      x);
}
```

4.4 Number of Unique Element in a Range

```
vector<pair<int, int>> Q[QQ];
int main() {
  int n; cin >> n;
  for(int i = 1; i <= n; i++) cin >> a[i];
  int q; cin >> q;
  for(int i = 1; i <= q; i++) {
    int l, r; cin >> l >> r;
    Q[r].push_back({l, i});
  }
  st.build(1, 1, n); // range sum and
      point add
  map<int, int> last_idx;
  for(int r = 1; r <= n; r++) {
    if(last_idx.find(a[r]) !=
        last_idx.end()) {
      st.upd(1, 1, n, last_idx[a[r]], -1);
    }
    last_idx[a[r]] = r;
    st.upd(1, 1, n, r, 1);
    for(auto i: Q[r]) {
      int l = i.first, id = i.second;
      ans[id] = st.query(1, 1, n, l, r);
    }
  }
}
```

4.5 Max Subarray Sum in a Range

```
struct node {
  ll tot_sum;
  ll prefix_max, suffix_max;
  ll max_subarray_sum;
};
node merge(node l, node r) {
  if (l.tot_sum == -inf) return r;
  if (r.tot_sum == -inf) return l;
  node ans;
  ans.max_subarray_sum =
      max(l.max_subarray_sum,
      r.max_subarray_sum);
  ans.max_subarray_sum =
      max(ans.max_subarray_sum,
      l.suffix_max + r.prefix_max);
  ans.tot_sum = l.tot_sum + r.tot_sum;
  ans.prefix_max = l.prefix_max;
  if (l.tot_sum + r.prefix_max >=
      l.prefix_max) {
    ans.prefix_max = l.tot_sum +
        r.prefix_max;
  }
  ans.suffix_max = r.suffix_max;
  if (r.tot_sum + l.suffix_max >=
      r.suffix_max) {
    ans.suffix_max = r.tot_sum +
        l.suffix_max;
  }
  return ans;
}
```

4.6 Count of subarrays such that their XOR is 0 in a range with upd

```
struct node {
  int cnt[64];
  node() {
    memset(cnt, 0, sizeof cnt);
  }
};
struct ST {
  node tree[4 * N]; int lazy[4 * N];
  void push(int n, int b, int e) {
    if (lazy[n] == 0) return;
    for (int k = 0; k < 64; k++) {
      if ((k ^ lazy[n]) > k) {
        swap(tree[n].cnt[k],
            tree[n].cnt[k ^ lazy[n]]);
      }
    }
    if (b != e) {
      int l = n << 1, r = l + 1;
      lazy[l] ^= lazy[n];
      lazy[r] ^= lazy[n];
    }
    lazy[n] = 0;
  }
  node merge(node a, node b) {
    for (int k = 0; k < 64; k++)
      a.cnt[k] += b.cnt[k];
    return a;
  }
  void build(int n, int  b, int e) {
    lazy[n] = 0;
    if (b == e) {
      tree[n].cnt[0] = 1;
      return;
    }
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    build(l, b, mid);
    build(r, mid + 1, e);
    tree[n] = merge(tree[l], tree[r]);
  }
  void upd(int n, int b, int e, int i,
      int j, int x) {
    push(n, b, e);
    if (b > j || e < i) return;
    if (b >= i && e <= j) {
      lazy[n] ^= x; push(n, b, e);
      return;
    }
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    upd(l, b, mid, i, j, x);
    upd(r, mid + 1, e, i, j, x);
    tree[n] = merge(tree[l], tree[r]);
  }
  node query(int n, int b, int e, int i,
      int j) {
    push(n, b, e);
    if (b > j || e < i) {
      node tmp;return tmp;
    }
    if (b >= i && e <= j) return tree[n];
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    node L = query(l, b, mid, i, j);
    node R = query(r, mid + 1, e, i, j);
    return merge(L, R);
  }
} st;
int32_t main() {
  string s; cin >> s;
  int n = s.size();
  s = '.' + s;
  st.build(1, 1, n);
  for (int i = 1; i <= n; i++) {
    int x = (1 << (s[i] - 'a'));
    st.upd(1, 1, n, i, n, x); // segtree
        contains prefix xor
  }
  int q; cin >> q;
  while (q--) {
    int type; cin >> type;
    if (type == 1) {
      int l, r; cin >> l >> r;
      node tmp = st.query(1, 1, n, l -
          1, r);
      ll ans = 0;
      for (int k = 0; k < 64; k++) {
        int x = tmp.cnt[k];
        if (l == 1 and k == 0) x++;
        ans += (1ll * x * (x - 1)) / 2;
      }
      cout << ans << '\n';
    }
    else {
      int i; cin >> i;
      char c; cin >> c;
      int mask = (1 << (s[i] - 'a'));
      mask ^= (1 << (c - 'a'));
      st.upd(1, 1, n, i, n, mask);
      s[i] = c;
    }
  }
}
```

4.7 Range Add, Set and Sum

```
// Problem: Range Updates and Sums - CSES
struct ST {
  int tree[4 * N];
  pair<int, int> lazy[4 * N]; // val,
      types
  // lazy[n].first = 0; default for all n
  // lazy[n].second = -1; default for all
      n
  void push(int n, int b, int e) {
    if (lazy[n].first == 0) return;
    if (lazy[n].second == 1) { // set
      tree[n] = lazy[n].first * (e - b +
          1);
    }
    else { // add
      tree[n] += lazy[n].first * (e - b
          + 1);
    }
    if (b != e) {
      int l = n << 1, r = l + 1;
      if (lazy[l].second == 1) {
        if (lazy[n].second == 1)
          lazy[l].first =
          lazy[n].first;
        else lazy[l].first +=
            lazy[n].first;
      }
      if (lazy[r].second == 1) {
        if (lazy[n].second == 1)
          lazy[r].first =
          lazy[n].first;
        else lazy[r].first +=
            lazy[n].first;
      }
      else {
        if (lazy[n].second == 1) {
          lazy[r].first = lazy[n].first;
          lazy[r].second = 1;
        }
        else lazy[r].first +=
            lazy[n].first;
      }
    }
    lazy[n].first = 0; lazy[n].second =
        -1;
  }
} st;
```

4.8 a[i] = a[i] * b + c in a Range

```
// Problem: Range Affine Range Sum
// lazy1[n] = 1; lazy2[n] = 0; // init in
    build
// lazy1[n] *= x; lazy2[n] += y; // when
    upd
mint tree[4 * N], lazy1[4 * N], lazy2[4
    * N];
void push(int n, int b, int e) {
  if (lazy1[n] == 1 or lazy2[n] == 0)
      return;
  tree[n] = tree[n] * lazy1[n] +
      lazy2[n] * (e - b + 1);
  if (b != e) {
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    lazy1[l] *= lazy1[n];
    lazy2[l] = lazy2[l] * lazy1[n] +
        lazy2[n];
    lazy1[r] *= lazy1[n];
    lazy2[r] = lazy2[r] * lazy1[n] +
        lazy2[n];
  }
  lazy1[n] = 1; lazy2[n] = 0;
}
```

4.9 [Trick] Historic Information - GSS2

```
// Statement: Max Subarray Sum in a Range
    but only Unique Value can contrib.
int ans[QQ];
vector<pair<int, int>> Q[QQ];
struct ST {
  int tree[4 * N], lazy[4 * N];
  int tree2[4 * N], lazy2[4 * N];
```

```
  void push(int n, int b, int e) {
    tree2[n] = max(tree2[n], tree[n] +
        lazy2[n]);
    tree[n] += lazy[n];
    if (b != e) {
      int l = n << 1, r = l + 1;
      lazy2[l] = max(lazy2[l], lazy[l] +
          lazy2[n]);
      lazy2[r] = max(lazy2[r], lazy[r] +
          lazy2[n]);
      lazy[l] += lazy[n];
      lazy[r] += lazy[n];
    }
    lazy[n] = 0; lazy2[n] = 0;
  }
  void upd(int n, int b, int e, int i,
      int j, int x) {
    push(n, b, e);
    if (b > j || e < i) return;
    if (b >= i && e <= j) {
      lazy[n] += x;
      lazy2[n] = max(lazy2[n], lazy[n]);
      push(n, b, e);
      return;
    }
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    upd(l, b, mid, i, j, x);
    upd(r, mid + 1, e, i, j, x);
    tree[n] = max(tree[l], tree[r]);
    tree2[n] = max(tree2[l], tree2[r]);
  }
  int query(int n, int b, int e, int i,
      int j) {
    push(n, b, e);
    if (b > j || e < i) return 0;
    if (b >= i && e <= j) return tree2[n];
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    int L = query(l, b, mid, i, j);
    int R = query(r, mid + 1, e, i, j);
    return max(L, R);
  }
} st;
int32_t main() {
  int n; cin >> n; int a[n + 1];
  for (int i = 1; i <= n; i++) cin >>
      a[i];
  int q; cin >> q;
  for (int i = 1; i <= q; i++) {
    int l, r; cin >> l >> r;
    Q[r].push_back({l, i});
  }
  map<int, int> last;
  for (int r = 1; r <= n; r++) {
    // for (int i = last[a[r]] + 1; i <=
        r; i++) {
    //   sum[i] += a[r];
    //   p[i] = max(p[i], sum[i]);
    // }
    st.upd(1, 1, n, last[a[r]] + 1, r,
        a[r]);
    last[a[r]] = r;
    for (auto i : Q[r]) {
      int l = i.first, id = i.second;
      ans[id] = st.query(1, 1, n, l, r);
    }
  }
}
```

4.10 [Problem] Strongest Community - LOJ

```
    // Given an Array, if any a_i = x
    exist all x are consecutive. Given
    [L, R] Q times, Print Max Freq.
struct node {
  int first_element, first_element_cnt;
  int last_element, last_element_cnt;
  int max_cnt;
};
node merge(node l, node r) {
  if(l.first_element == -1) return r;
  if(r.first_element == -1) return l;
  node ans;
  ans.max_cnt = max(l.max_cnt, r.max_cnt);
  if(l.last_element == r.first_element) {
    ans.max_cnt = max(ans.max_cnt,
        l.last_element_cnt +
        r.first_element_cnt);
  }
  ans.first_element = l.first_element;
  ans.first_element_cnt =
      l.first_element_cnt;
  if(l.first_element == r.first_element) {
    ans.first_element_cnt +=
        r.first_element_cnt;
  }
  ans.last_element = r.last_element;
  ans.last_element_cnt =
      r.last_element_cnt;
  if(r.last_element == l.last_element) {
    ans.last_element_cnt +=
        l.last_element_cnt;
  }
  return ans;
}
```

4.11 [Problem] Diablo - LOJ

```
// Add element at the End, and Remove Kth
    Index (also print it's value)
int deleted_idx;
struct ST {
  pair<int, int> tree[4 * (N + Q)]; //
      sum, alive_cnt
  pair<int, int> query(int n, int b, int
      e, int x) {
    if (b > e) return { -1, -1};
    if (tree[n].second < x) return
        {tree[n].second, -1};
    if (b == e) {
      deleted_idx = b;
      return tree[n];
    }
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    pair<int, int> L = query(l, b, mid,
        x);
    if (L.second != -1) return L;
    pair<int, int> R = query(r, mid + 1,
        e, x - L.first);
    return R;
  }
} st;
void solve() {
  int id = n, end = n + q + 5;
  while (q--) {
    char c; cin >> c;
    if (c == 'a') {
      int p; cin >> p;
      st.upd(1, 1, end, ++id, p, 1);
    }
    else {
      int k; cin >> k;
      pair<int, int> ans = st.query(1,
          1, end, k);
      if (ans.second == -1) cout <<
          "none\n";
      else {
        cout << ans.first << '\n';
        st.upd(1, 1, end, deleted_idx,
            0, 0);
      }
    }
  }
}
```

4.12 Merge Sort Tree with Point Upd

```
struct MST { // merge sort tree
  o_set<array<int, 2>> tree[4 * N]; //
      greater<T>
  o_set<array<int, 2>>
      merge(o_set<array<int, 2>> &a,
      o_set<array<int, 2>> &b) {
    int i = 0, j = 0;
    int n = a.size(), m = b.size();
    o_set<array<int, 2>> ans;
    for (auto x : a) ans.insert(x);
    for (auto x : b) ans.insert(x);
    return ans;
  }
  void build(int n, int b, int e) {
    if (b == e) {
      tree[n].insert({a[b], b});
      return;
    }
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    build(l, b, mid);
    build(r, mid + 1, e);
    tree[n] = merge(tree[l], tree[r]);
  }
  void upd(int n, int b, int e, int i,
      int x) {
    if (b == e) {
      tree[n].erase({a[b], b});
      tree[n].insert({x, b});
      a[b] = x;
      return;
    }
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    tree[n].erase({a[i], i});
    tree[n].insert({x, i});
    if (i <= mid) upd(l, b, mid, i, x);
    else upd(r, mid + 1, e, i, x);
  }
  int query(int n, int b, int e, int i,
      int j, int k) {
    if (b > j || e < i) return 0;
    if (b >= i && e <= j) {
      int idx = tree[n].order_of_key({k,
          inf});
      return idx;
    }
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    int L = query(l, b, mid, i, j, k);
    int R = query(r, mid + 1, e, i, j, k);
    return (L + R);
  }
} mst;
```

4.13 Sparse Table

```
const int N = 2e5 + 9, K = 20; // change
    here
int a[N], tree[N][K];
int log2_floor(unsigned long long i) {
  return i ? __builtin_clzll(1) -
      __builtin_clzll(i) : -1;
}
void build(int n) {
  for (int i = 1; i <= n; i++) {
    tree[i][0] = a[i];
  }
  for (int k = 1; k < K; k++) {
    for (int i = 1; i + (1 << k) - 1 <=
        n; i++) {
      tree[i][k] = min(tree[i][k - 1],
          tree[i + (1 << (k - 1))][k -
          1]); // change here
    }
  }
}
int query(int l, int r) {
  int k = log2_floor(r - l + 1);
  return min(tree[l][k], tree[r - (1 <<
      k) + 1][k]); // change here
}
build(n);
```

4.14 DSU

```
struct DSU {
  vector<int> par, sz;
  int c;
  DSU(int n) {
    par.resize(n + 1), sz.resize(n + 1,
        1);
    for (int i = 1; i <= n; i++) {
      par[i] = i;
    }
    c = n;
  }
  int find(int i) {
    return (i == par[i] ? i : par[i] =
        find(par[i]));
  }
  bool same(int i, int j) {
```

```
    return find(i) == find(j);
  }
  int get_size(int i) {
    return sz[find(i)];
  }
  int count() { // number of connected
      components
    return c;
  }
  int merge(int i, int j) {
    if ((i = find(i)) == (j = find(j)))
        return -1;
    c--;
    if (sz[i] < sz[j]) swap(i, j);
    par[j] = i;
    sz[i] += sz[j];
    return i;
  }
};
DSU dsu(n);
```

4.15 MOs Algorithm

```
const int N = 1e6 + 9, B = 440;
struct query {
  int l, r, id;
  bool operator < (const query &x) const {
    if (l / B == x.l / B) return ((l /
        B) & 1) ? r > x.r : r < x.r;
    return l / B < x.l / B;
  }
} Q[N];
int cnt[N], a[N];
long long sum;
inline void add_left(int i) {
  int x = a[i];
  sum += 1LL * (cnt[x] + cnt[x] + 1) * x;
  ++cnt[x];
}
inline void add_right(int i) {
  int x = a[i];
  sum += 1LL * (cnt[x] + cnt[x] + 1) * x;
  ++cnt[x];
}
inline void rem_left(int i) {
  int x = a[i];
  sum -= 1LL * (cnt[x] + cnt[x] - 1) * x;
  --cnt[x];
}
inline void rem_right(int i) {
  int x = a[i];
  sum -= 1LL * (cnt[x] + cnt[x] - 1) * x;
  --cnt[x];
}
long long ans[N];
int32_t main() {
  int n, q; cin >> n >> q;
  for (int i = 1; i <= n; i++) cin >>
      a[i];
  for (int i = 1; i <= q; i++) {
    cin >> Q[i].l >> Q[i].r;
    Q[i].id = i;
  }
  sort(Q + 1, Q + q + 1);
  int l = 1, r = 0;
  for (int i = 1; i <= q; i++) {
    int L = Q[i].l, R = Q[i].r;
    if (R < l) {
      while (l > L) add_left(--l);
      while (l < L) rem_left(l++);
      while (r < R) add_right(++r);
      while (r > R) rem_right(r--);
    } else {
      while (r < R) add_right(++r);
      while (r > R) rem_right(r--);
      while (l > L) add_left(--l);
      while (l < L) rem_left(l++);
    }
    ans[Q[i].id] = sum;
  }
  for (int i = 1; i <= q; i++) cout <<
      ans[i] << '\n';
}
```

4.16 MOs with Updates

```
const int N = 2e5 + 9;
const int B = 2500;
struct query {
  int l, r, t, id;
  bool operator < (const query& x) const {
    if (l / B == x.l / B) {
      if (r / B == x.r / B) return t <
          x.t;
      return r / B < x.r / B;
    }
    return l / B < x.l / B;
  }
} Q[N];
struct upd {
  int pos, old, cur;
} U[N];
int a[N], cnt[N], f[N], ans[N], l, r, t;
int sum = 0, now[N];
inline void add(int x) {
  int val = now[x];
  if ( val % 3 != 0 ) val = 0;
  if ( cnt[x] == 0 ) sum += val;
  ++cnt[x];
}
inline void del(int x) {
  int val = now[x];
  if ( val % 3 != 0 ) val = 0;
  --cnt[x];
  if ( cnt[x] == 0 ) sum -= val;
}
inline void update(int pos, int x) {
  if (l <= pos && pos <= r) {
    add(x);
    del(a[pos]);
  }
  a[pos] = x;
}
map <int, int> mp;
int nxt = 0;
int get(int x) {
  if ( !mp.count(x) ) {
    mp[x] = nxt; now[nxt] = x;
    nxt++;
  }
  return mp[x];
}
void AmeDoko() {
  int n, q; cin >> n >> q;
  for (int i = 1; i <= n; i++) {
    cin >> a[i];
    a[i] = get(a[i]);
  }
  int nq = 0, nu = 0;
  for (int i = 1; i <= q; i++) {
    int ty, l, r;
    cin >> ty >> l >> r;
    if (ty == 1) ++nq, Q[nq] = { l, r,
        nu, nq };
    else ++nu, U[nu].pos = l, U[nu].old
        = a[l], a[l] = get(r), U[nu].cur
        = a[l];
  }
  sort(Q + 1, Q + nq + 1);
  t = nu, l = 1, r = 0;
  for (int i = 1; i <= nq; i++) {
    int L = Q[i].l, R = Q[i].r, T =
        Q[i].t;
    while (t < T) t++, update(U[t].pos,
        U[t].cur);
    while (t > T) update(U[t].pos,
        U[t].old), t--;
    if (R < l) {
      while (l > L) add(a[--l]);
      while (l < L) del(a[l++]);
      while (r < R) add(a[++r]);
      while (r > R) del(a[r--]);
    }
    else {
      while (r < R) add(a[++r]);
      while (r > R) del(a[r--]);
      while (l > L) add(a[--l]);
      while (l < L) del(a[l++]);
    }
    ans[Q[i].id] = sum;
  }
  for (int i = 1; i <= nq; i++) cout <<
      ans[i] << '\n';
}
```

4.17 Wavelet Tree

```
struct wavelet_tree {
  int low, high;
  wavelet_tree *lft = NULL, *rgt = NULL;
  int *pref = NULL;
  long long *sum = NULL; // remove if no
      need
  wavelet_tree(int *l, int *r, int low,
      int high): low(low), high(high) {
    if (l >= r || low >= high) return;
    pref = new int[r - l + 2];
    sum = new long long[r - l + 2];
    pref[0] = 0;
    sum[0] = 0;
    int mid = (low + high) >> 1, cnt = 1;
    for (int *i = l; i != r; i++, cnt++) {
      pref[cnt] = pref[cnt - 1] + ((*i)
          <= mid);
      sum[cnt] = sum[cnt - 1] + (*i);
    }
    int *pivot = stable_partition(l, r,
        [&](int x) {return x <= mid;});
    lft = new wavelet_tree(l, pivot,
        low, mid);
    rgt = new wavelet_tree(pivot, r, mid
        + 1, high);
  }
  // swaps the elements at index 'idx'
      and 'idx+1'
  void swap_adjacent(int idx) {
    if (low == high)return;
    int firstBit = pref[idx] - pref[idx
        - 1];
    sum[idx] = sum[idx - 1] + sum[idx +
        1] - sum[idx];
    if (firstBit == (pref[idx + 1] -
        pref[idx])) {
      if (firstBit)
        lft->swap_adjacent(pref[idx]);
      else rgt->swap_adjacent(idx -
          pref[idx]);
    }
    else {
      if (firstBit) pref[idx]--;
      else pref[idx]++;
    }
  }
  // count occurences of 'k' in range
      [l,r]
  int count(int l, int r, int k) {
    if (l > r || high < k || low > k)
        return 0;
    if (low == high) return r - l + 1;
    int mid = (low + high) >> 1;
    if (k <= mid) return
        lft->count(pref[l - 1] + 1,
        pref[r], k);
    return rgt->count(l - pref[l - 1], r
        - pref[r], k);
  }
  // returns the k'th smallest element in
      range [l,r]  // act like multiset
  int kth(int l, int r, int k) {
    if (l > r) return 0;
    if (low == high) return low;
    int lftCount = pref[r] - pref[l - 1];
    if (lftCount >= k) return
        lft->kth(pref[l - 1] + 1,
        pref[r], k);
    return rgt->kth(l - pref[l - 1], r -
        pref[r], k - lftCount);
  }
  // returns the count of elements that
      are less than or equal to 'k' in
      range [l,r]
  int LTE(int l, int r, int k) {
    if (l > r || low > k) return 0;
    if (high <= k) return r - l + 1;
    return lft->LTE(pref[l - 1] + 1,
        pref[r], k) + rgt->LTE(l -
        pref[l - 1], r - pref[r], k);
  }
  // returns the count of elements that
      are greater than or equal to 'k' in
      range [l,r]
```

```
  int GTE(int l, int r, int k) {
    if (l > r || high < k) return 0;
    if (low >= k) return r - l + 1;
    return lft->GTE(pref[l - 1] + 1,
        pref[r], k) + rgt->GTE(l -
        pref[l - 1], r - pref[r], k);
  }
  // returns the sum of elements less
      than 'k' in range [l,r]
  long long sum_query(int l, int r, int
      k) {
    if (l > r || low >= k) return 0;
    if (low == high) return 1LL * (r - l
        + 1) * low;
    if (high < k) return sum[r] - sum[l
        - 1];
    return lft->sum_query(pref[l - 1] +
        1, pref[r], k) +
        rgt->sum_query(l - pref[l - 1],
        r - pref[r], k);
  }
  ~wavelet_tree() {
    if (pref != NULL) delete []pref;
    if (sum != NULL) delete []sum;
    if (lft != NULL) delete lft;
    if (rgt != NULL) delete rgt;
  }
};
int32_t main() {
  int n; cin >> n;
  for (int i = 1; i <= n; i++) cin >>
      a[i];
  wavelet_tree t = wavelet_tree(a + 1, a
      + n + 1, -MAXV, MAXV); // a[i] will
      change
}
```

4.18 Persistent Segment Tree

```
struct node {
  int cnt; ll val; // change here
  node *lc, *rc;
};
node *versions[N];
vector<node *> nodes;
node* build(int b, int e) {
  node *me = new node();
  nodes.push_back(me);
  if (b == e) {
    me->cnt = 0; // change here
    me->val = 0; // change here
    me->lc = NULL;
    me->rc = NULL;
    return me;
  }
  int mid = (b + e) >> 1;
  me->lc = build(b, mid);
  me->rc = build(mid + 1, e);
  me->cnt = me->lc->cnt + me->rc->cnt; //
      change here
  me->val = me->lc->val + me->rc->val; //
      change here
  return me;
}
node *upd(node *me, int b, int e, int i,
    int x, int y) {
  node *newme = new node();
  nodes.push_back(newme);
  if (b == e and b == i) {
    newme->cnt = me->cnt + x; // change
        here
    newme->val = me->val + y; // change
        here
    newme->lc = NULL;
    newme->rc = NULL;
    return newme;
  }
  int mid = (b + e) >> 1;
  if (i <= mid) {
    newme->lc = upd(me->lc, b, mid, i,
        x, y);
    newme->rc = me->rc;
  }
  else {
    newme->rc = upd(me->rc, mid + 1, e,
        i, x, y);
    newme->lc = me->lc;
  }
  newme->cnt = newme->lc->cnt +
      newme->rc->cnt; // change here
  newme->val = newme->lc->val +
      newme->rc->val; // change here
  return newme;
}
// returns kth element in range [l, r]
    (range provided when call)
int kth(node *me1, node *me2, int b, int
    e, int k) {
  if (b == e) {
    return b;
  }
  int cnt = me1->lc->cnt - me2->lc->cnt;
  int mid = (b + e) >> 1;
  if (cnt >= k) {
    return kth(me1->lc, me2->lc, b, mid,
        k);
  }
  return kth(me1->rc, me2->rc, mid + 1,
      e, k - cnt);
}
// returns the count of elements that are
    between [x, y] in range [l,r]
int count(node *me1, node *me2, int b,
    int e, int x, int y) {
  if (b > y or e < x) return 0; // return
      appropriate value
  if (b >= x and e <= y) return me1->cnt
      - me2->cnt; // change here
  int mid = (b + e) >> 1;
  int L = count(me1->lc, me2->lc, b,
      mid, x, y);
  int R = count(me1->rc, me2->rc, mid +
      1, e, x, y);
  return (L + R);
}
// return the sum of values between [i,
    j] and range [l, r] (provided when
    call)
ll sum_query(node *me1, node *me2, int
    b, int e, int i, int j) {
  if (b > j or e < i) return 0; // return
      appropriate value
  if (b >= i and e <= j) return me1->val
      - me2->val; // change here
  int mid = (b + e) >> 1;
  ll L = sum_query(me1->lc, me2->lc, b,
      mid, i, j);
  ll R = sum_query(me1->rc, me2->rc, mid
      + 1, e, i, j);
  return (L + R);
}
// Caution: Understand which value to
    used (Real or Compressed)
int32_t main() {
  // must clear memory
  for (node * me : nodes) delete me;
  nodes.clear();
}
```

4.19 Trie for bit

```
struct Trie {
  static const int B = 31;
  struct node {
    node* nxt[2];
    int sz;
    node() {
      nxt[0] = nxt[1] = NULL;
      sz = 0;
    }
  }*root;
  Trie() {
    root = new node();
  }
  void insert(int val) {
    node* cur = root;
    cur -> sz++;
    for (int i = B - 1; i >= 0; i--) {
      int b = val >> i & 1;
      if (cur -> nxt[b] == NULL) cur ->
          nxt[b] = new node();
      cur = cur -> nxt[b];
      cur -> sz++;
    }
  }
  void erase( int val ) {
    node *cur = root;
    cur->sz--;
    for (nt i = B - 1; i >= 0; i--) {
      int x = val >> i & 1;
      if (cur->nxt[x] == NULL) return;
      cur = cur->nxt[x];
      cur->sz--;
    }
  }
   int query(int x, int k) {  // number
      of values s.t. val ⊕ x < k
    node* cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
      if (cur == NULL) break;
      int b1 = x >> i & 1, b2 = k >> i &
          1;
      if (b2 == 1) {
        if (cur -> nxt[b1]) ans += cur
            -> nxt[b1] -> sz;
        cur = cur -> nxt[!b1];
      } else cur = cur -> nxt[b1];
    }
    return ans;
  }
   int get_max(int x) {  // returns
      maximum of val ⊕ x
    node* cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
      int k = x >> i & 1;
      if (cur -> nxt[!k]) cur = cur ->
          nxt[!k], ans <<= 1, ans++;
      else cur = cur -> nxt[k], ans <<= 1;
    }
    return ans;
  }
   int get_min(int x) {  // returns
      minimum of val ⊕ x
    node* cur = root;
    int ans = 0;
    for (int i = B - 1; i >= 0; i--) {
      int k = x >> i & 1;
      if (cur -> nxt[k]) cur = cur ->
          nxt[k], ans <<= 1;
      else cur = cur -> nxt[!k], ans <<=
          1, ans++;
    }
    return ans;
  }
  void del(node* cur) {
    for (int i = 0; i < 2; i++) if (cur
        -> nxt[i]) del(cur -> nxt[i]);
    delete(cur);
  }
} t;
```

4.20 2D BIT

```
const int N = 2500 + 10;
int a[N][N], tree[N][N];
void update(int x, int y, int val) { // x
    and y is the coordinate from range
    update
  for (int i = x; i < N; i += i & (-i)) {
    for (int j = y; j < N; j += j &
        (-j)) {
      tree[i][j] += val;
    }
  }
}
int query(int x, int y) {
  int res = 0;
  for (int i = x; i < N; i += i & (-i)) {
    for (int j = y; j < N; j += j &
        (-j)) {
      res += tree[i][j];
    }
  }
  return res;
}
void range_add(int a, int b, int c, int
    d, int val) {
  update(a, b, val);
```

```
  update(a, d + 1, -val);
  update(c + 1, b, -val);
  update(c + 1, d + 1, val);
}
```

4.21 Treap

```
mt19937 rnd(chrono::steady_clock::now().
    time_since_epoch().count());
struct node {
  int prior, size;
  int val, sum;
  node *l, *r;
  node(int value) {
    l = r = nullptr;
    val = value;
    sum = value;
    size = 1;
    prior = rnd();
  }
};
typedef node* pnode;
int sz(pnode t) {
  if (t == nullptr) {return 0;}
  return t->size;
}
void upd_sz(pnode t) {
  if (t != nullptr) {
    t->size = sz(t->l) + 1 + sz(t->r);
  }
}
////////////////////based on index
void reverse_check(pnode t) {
  if (t == nullptr) {return;}
  if (t->inv == 1) {
    swap(t->l, t->r);
    if (t->l != nullptr) {t->l->inv ^= 1;}
    if (t->r != nullptr) {t->r->inv ^= 1;}
    t->inv = 0;
  }
}

void fix_node_info(pnode& t, pnode l,
    pnode r) {
  if (!l || !r)return; // void(t =
      l?l:r);
  t->sum = l->sum + r->sum;
}

void update_node_info(pnode t) {
  if (!t) {return;}
  //reverse_check(t);
  t->sum = t->val;      //reset node
  fix_node_info(t, t->l, t);
  fix_node_info(t, t, t->r);
}

void split(pnode t, pnode &l, pnode &r,
    int pos, int add = 0) {
  if (!t) {l = r = nullptr; return;}
  //reverse_check(t);
  int curr_pos = add + sz(t->l);
  if (curr_pos <= pos) {
    split(t->r, t->r, r, pos, curr_pos +
        1); l = t;
  }
  else {
    split(t->l, l, t->l, pos, add); r = t;
  }
  upd_sz(t);
  update_node_info(t);
}
void merge(pnode &t, pnode l, pnode r) {
  //reverse_check(t);
  //reverse_check(l);
  //reverse_check(r);
  if (!l || !r) {t = l ? l : r;}
  else if (l->prior > r->prior) {
    merge(l->r, l->r, r);
    t = l;
  }
  else {
    merge(r->l, l, r->l);
    t = r;
  }
  upd_sz(t);
  //update_node_info(t);
}
////////////////////////////based on
    value
void split(pnode t, pnode &l, pnode &r,
    int key) {
  if (!t) {l = r = nullptr;}
  else if (t->val <= key) {
    split(t->r, t->r, r, key);
    l = t;
  }
  else {
    split(t->l, l, t->l, key);
    r = t;
  }
  upd_sz(t);
}
void merge_treap(pnode &t, pnode l,
    pnode r) {
  if (!l || !r) {
    t = l ? l : r;
    return;
  }
  if (l->prior < r->prior) {swap(l, r);}
  node *L, *R;
  split(r, L, R, l->val);
  merge_treap(l->r, l->r, R);
  merge_treap(l->l, L, l->l);
  t = l;
  upd_sz(t);
}
void erase(pnode &t, pnode &erase_part,
    int L, int R) {
  pnode l, r, mid, mr;
  split(t, l, r, L - 1);
  split(r, mid, mr, R);
  merge(t, l, mr);
  erase_part = mid;
  upd_sz(t);
}
void erase(pnode &t, int value) {
  if (!t) {return;}
  else if (t->val == value) {
    pnode temp = t;
    merge(t, t->l, t->r);
    free(temp);
  }
  else if (t->val < value) {
    erase(t->r, value);
  }
  else {
    erase(t->l, value);
  }
  upd_sz(t);
}
void output(pnode t) {
  if (t == nullptr) {return;}
  output(t->l);
  cout << t->val << " ";
  output(t->r);
}
//main function
/*
pnode root=nullptr;
merge(root,root, new node(354));
    //inserting
*/
```

4.22 Centroid Decomposition

```
set<int>graph[N];
int par[N];
int sz[N];
int ans[N];
int nn = 0;
void dfs_sz(int x, int p) {
  sz[x] = 1;
  nn++;
  for (auto u : graph[x]) {
    if (u != p) {
      dfs_sz(u, x);
      sz[x] += sz[u];
    }
  }
}
int dfs_centroid(int x, int p) {
  for (auto u : graph[x]) {
    if (u != p && sz[u] > nn / 2) {
      return dfs_centroid(u, x);
    }
  }
  return x;
}
void decompose(int x, int p) {
  nn = 0;
  dfs_sz(x, x);
  int centroid = dfs_centroid(x, x);
  if (p == -1) {p = centroid;}
  par[centroid] = p;
  for (auto u : graph[centroid]) {
    graph[u].erase(centroid);
    decompose(u, centroid);
  }
  graph[centroid].clear();
}
int query(int node) {
  int x = node;
  int res = LLMax;
  while (1) {
    res = min(res, find_dis(node, x) +
        ans[x]);
    if (x == par[x]) {break;}
    x = par[x];
  }
  return res;
}
void update(int node) {
  int x = node;
  while (1) {
    ans[x] = min(ans[x], find_dis(node,
        x));
    if (x == par[x]) {break;}
    x = par[x];
  }
}
```

5 Dynamic Programming

5.1 Knapsack 2

```
Constraints: N <= 100, W <= 1e9, val[i]
    <= 1000
dp[i][cur_val] = min weight needed to
    achieve cur_val from i to n. if
    dp[1][val] <= W then val is a
    candidate ans. where val is
    all_possible_val (1 to 100*1000)
```

5.2 LIS using Segment Tree

```
int32_t main() {
  int n; cin >> n;
  for (int i = 1; i <= n; i++) {
    cin >> a[i]; // a[i] must be >= 2
  }
  // dp[i] = LIS ending at pos i
  st.build(1, 1, M); // range max query,
      and upd idx with max(cur_val,
      new_val)
  for (int i = 1; i <= n; i++) {
    dp[i] = 1;
    if (a[i] != 1) {
      int mx = st.query(1, 1, M, 1, a[i]
          - 1);
      mx++;
      dp[i] = max(dp[i], mx);
    }
    st.upd(1, 1, M, a[i], dp[i]);
  }
  int ans = 0;
  for (int i = 1; i <= n; i++) ans =
      max(ans, dp[i]);
  cout << ans << '\n';
}
```

5.3 Digit DP

```
// Problem: How Many Zeroes? - LightOJ
  // How many zeroes between n to m
    (n <= m). 100 to 102 ans 4
ll fun2(int i, bool is_small) {
  if (i == sz) return 1;
  int l = 0, r = s[i] - '0';
  if (is_small) r = 9;
  ll &ans = dp2[i][is_small];
  if (ans != -1) return ans;
  ans = 0;
```

```
  for (int x = l; x <= r; x++) {
    ans += fun2(i + 1, (is_small | (x <
        r)));
  }
  return ans;
}
ll fun(int i, bool is_small, bool
    has_started) {
  if (i == sz) return 0;
  int l = 0, r = s[i] - '0';
  if (is_small) r = 9;
  ll &ans = dp1[i][is_small][has_started];
  if (ans != -1) return ans;
  ans = 0;
  for (int x = l; x <= r; x++) {
    bool new_has_started = has_started |
        (x != 0);
    ans += fun(i + 1, (is_small | (x <
        r)), new_has_started);
    if (x == 0 and has_started) {
      ans += fun2(i + 1, (is_small | (x
          < r)));
    }
  }
  return ans;
}
void get(long long x) {
  if (x < 0) return; s = "";
  while (x > 0) {
    char c = (x % 10) + '0';
    s += c; x /= 10;
  }
  reverse(s.begin(), s.end());
  sz = s.size();
  memset(dp1, -1, sizeof(dp1));
  memset(dp2, -1, sizeof(dp2));
}
void solve() {
  ll n, m; cin >> n >> m;
  get(n - 1);
  memset(dp1, -1, sizeof(dp1));
  memset(dp2, -1, sizeof(dp2));
  ll ans1 = (n == 0) ? 0 : fun(0, false,
      false);
  get(m);
  ll ans2 = fun(0, false, false);
  cout << ans2 - ans1 + (n == 0) << '\n';
}
```

5.4 Bitmask DP

```
// Problem: DNA Sequence - LOJ
// Given n strings, find the shortest
    string that contains all the given
    strings as substrings and
    lexicographically smallest.
const int N = 17;
int n, tail[N][N], dp[(1 << N) + 2][N +
    2];
string s[N + 2];
bool cmp(string a, string b) {
  if (a.size() < b.size()) {
    return true;
  }
  return false;
}
int fun(int mask, int last) {
  if (__builtin_popcount(mask) >= n)
      return 0;
  int &ans = dp[mask][last];
  if (ans != -1) return ans;
  ans = 1e9;
  for (int j = 1; j <= n; j++) {
    if (!(mask & (1 << j))) {
      if (last == n + 1) {
        int x = s[j].length() + fun(mask
            | (1 << j), j);
        ans = min(ans, x);
      }
      else {
        int x = (s[j].length() -
            tail[last][j]) + fun(mask |
            (1 << j), j);
        ans = min(ans, x);
      }
    }
  }
  return ans;
}
void print(int mask, int last) {
  if (__builtin_popcount(mask) >= n)
      return;
  int ans = fun(mask, last), idx = -1;
  string str = "{";
  for (int j = 1; j <= n; j++) {
    if (!(mask & (1 << j))) {
      if (last == n + 1) {
        int x = s[j].length() + fun(mask
            | (1 << j), j);
        if (x == ans) {
          string d = s[j];
          if (d <= str) str = d; idx = j;
        }
      }
      else {
        int x = (s[j].length() -
            tail[last][j]) + fun(mask |
            (1 << j), j);
        if (x == ans) {
          string d =
              s[j].substr(tail[last][j]);
          if (d <= str) str = d; idx = j;
        }
      }
    }
  }
  cout << s[idx].substr(tail[last][idx]);
  print(mask | (1 << idx), idx);
}
void solve() {
  cin >> n;
  string str[n + 1];
  for (int i = 1; i <= n; i++) {
    cin >> str[i]; // len <= 100
  }
  // remove duplicates and strings which
      is a subarray of others
  sort(str + 1, str + n + 1, cmp);
  vector<string> vec;
  for (int i = 1; i <= n; i++) {
    bool ok = true;
    for (int j = i + 1; j <= n; j++) {
      if (str[j].find(str[i]) !=
          string::npos) {
        ok = false;
        break;
      }
    }
    if (ok) vec.push_back(str[i]);
  }
  n = vec.size();
  int idx = 1;
  for (auto x : vec) s[idx++] = x;
  // maximum length that suffix of a[i] =
      prefix of a[j]
  memset(tail, 0, sizeof(tail));
  for (int i = 1; i <= n; i++) {
    string x = s[i];
    for (int j = 1; j <= n; j++) {
      string y = s[j];
      int cnt = 0;
      for (int k = min(x.length(),
          y.length()); k >= 1; k--) {
        if (x.substr(x.length() - k) ==
            y.substr(0, k)) {
          cnt = k;
          break;
        }
      }
      tail[i][j] = cnt;
    }
  }
  memset(dp, -1, sizeof(dp));
  fun(0, n + 1);
  print(0, n + 1);
}
```

5.5 MCM DP

```
// Problem: Slimes - Atcoder DP Contest
// Given n slimes. Choose two adjacent
    slimes, and combine them into a new
    slime. The new slime has a size of
    x+y, where x and y are the sizes of
    the slimes before combining them.
    Here, a cost of x+y is incurred.
    Example-
// (10, 20, 30, 40) → (30, 30, 40)
// (30, 30, 40) → (60, 40)
// (60, 40) → (100) ans = 190
        // Solution: Think reverse. We
    are given the final sum, from i to j.
    Now we will cut any point between i to
    j and calculate the cost
// Time: O(n^3)
ll fun(int i, int j) {
  if (i == j) return 0;
  ll &ans = dp[i][j];
  if (ans != -1) return ans;
  ll cur = 0;
  for (int x = i; x <= j; x++) {
    cur += a[x];
  }
  ans = inf;
  for (int x = i; x < j; x++) {
    ans = min(ans, cur + fun(i, x) +
        fun(x + 1, j));
  }
  return ans;
}
cout << fun(1, n) << '\n';
```

5.6 Number of Unique Subsequence

```
int nxt[N][26], dp[N];
int f(int i) {
  if (i > n) return 0;
  int &ans = dp[i];
  if (ans != -1) return ans;
  ans = 1;
  for (int c = 0; c < 26; c++) {
    ans = (ans + f(nxt[i][c])) % mod;
  }
  return ans;
}
// nxt[i][c] = nxt c strictly after i
void solve() {
  cin >> s; s = '.' + s;
  cout << f(0) << '\n';
}
```

5.7 [Problem] Rose Land - SUST 24

```
        // Given a complete
    binary tree, i-th node grows a_i roses
    per day and deliver to u in dis(u,i)
    days. Alice at node u, How many roses
    he will get within t days? u and t for
    Q queries.
    // Idea: as it is a complete binary
    tree, max dis is 2*logn. We can do DP
    on Tree for ~40 days.
ll dp[N][45], dp2[N][45];
void dfs(int u, int p) {
  for (int i = 1; i <= 40; i++) {
    dp[u][i] = i * a[u];
  }
  for (auto v : g[u]) {
    if (v != p) {
      dfs(v, u);
      for (int i = 1; i <= 40; i++) {
        dp[u][i] += dp[v][i - 1];
      }
    }
  }
}
void dfs2(int u, int p) {
  for (int i = 2; u != 1 and i <= 40;
      i++) {
    dp2[u][i] = (dp[p][i - 1] + dp2[p][i
        - 1]) - dp[u][i - 2];
  }
  for (auto v : g[u]) {
    if (v != p) {
      dfs2(v, u);
    }
  }
}
int32_t main() {
  cin >> n; ll sum = 0;
  for (int i = 1; i <= n; i++) {
```

```
    cin >> a[i]; sum += a[i];
  }
  dfs(1, 0); // rose from subtree
  dfs2(1, 0); // rose from par
  int q; cin >> q;
  while (q--) {
    int u, t; cin >> u >> t;
    int extra = t - 40;
    t = min(40, t);
    ll ans = dp[u][t] + dp2[u][t];
    if (extra > 0) ans += 1ll * extra *
        sum;
    cout << ans << '\n';
  }
}
```

5.8 DP with DS

```
// Problem: Save The Trees - LOJ
// Given N trees in order, each with
    height and type, partition them into
    groups so that every tree is used, no
    group contains duplicate types, each
    groups cost is its tallest trees
    height, minimize total cost.
      // helper[j] = dp[j - 1] + rangemax(j,i);
    (i to j in same group)
    // dp[i] = min(helper[j]) for all
    possible valid j
const int N = 2e5 + 9, inf = 1e18;
int a[N], b[N], n, lft_idx[N], dp[N];
map<int, int> last_pos;
void solve() {
  cin >> n;
  for (int i = 1; i <= n; i++) {
    cin >> a[i] >> b[i]; // type, height
  }
  for (int i = 1; i <= n; i++) {
    if (last_pos.find(a[i]) ==
        last_pos.end()) lft_idx[i] = 1;
    else lft_idx[i] = last_pos[a[i]] + 1;
    lft_idx[i] = max(lft_idx[i],
        lft_idx[i - 1]);
    last_pos[a[i]] = i;
  }
  // segtree range min query and range
      add upd
  st1.build(1, 1, n);
  st2.build(1, 1, n);
  memset(dp, 0, sizeof dp);
  stack<pair<int, int>> stk;
  stk.push({inf, 0});
  for (int i = 1; i <= n; i++) {
    int x = b[i], lft = lft_idx[i], pos
        = i;
    while (!stk.empty() and
        stk.top().second >= lft and
        stk.top().first <= x) {
      int mx = st2.query(1, 1, n,
          stk.top().second + 1, pos);
      st2.upd(1, 1, n, stk.top().second
          + 1, pos, -mx);
      st2.upd(1, 1, n, stk.top().second
          + 1, pos, x);
      st1.upd(1, 1, n, stk.top().second
          + 1, pos, -mx);
      st1.upd(1, 1, n, stk.top().second
          + 1, pos, x);
      pos = stk.top().second;
      stk.pop();
    }
    int mx = st2.query(1, 1, n,
        stk.top().second + 1, pos);
    st2.upd(1, 1, n, stk.top().second +
        1, pos, -mx);
    st2.upd(1, 1, n, stk.top().second +
        1, pos, x);
    st1.upd(1, 1, n, stk.top().second +
        1, pos, -mx);
    st1.upd(1, 1, n, stk.top().second +
        1, pos, x);
    stk.push({x, i});
    dp[i] = st1.query(1, 1, n, lft, i);
    if (i + 1 <= n) st1.upd(1, 1, n, i +
        1, i + 1, dp[i]);
  }
}
```

5.9 [Problem] Sub-Palindromic Tree

```
// Given a tree, each node has a char.
    Print max len subsequence any path
    which is a palindrome. (CF/1771/D)
// Time: O(N^2)
int nxt[N][N], dp[N][N];
vector<int> vec;
void dfs(int u, int p) {
  vec.push_back(u);
  for (auto v: g[u]) {
    if (v != p) dfs(v, u);
  }
}
int f(int u, int v) {
  if (v == u) return 1;
  int &ans = dp[u][v];
  if (ans != -1) return ans;
  ans = 0;
  if (s[u] == s[v]) ans = 2 + (nxt[u][v]
      == v ? 0 : f(nxt[u][v],
      nxt[v][u]));
  else ans = max(f(nxt[u][v], v), f(u,
      nxt[v][u]));
  return ans;
}
void solve() {
  cin >> n >> s; s = '.' + s;
  for (int i = 1; i < n; i++) {
    int u, v; cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u);
  }
          // nxt[u][v] = next node after u
      if I want to go from u to v
  for (int u = 1; u <= n; u++) {
    for (auto x : g[u]) {
      vec.clear();
      dfs(x, u);
      for (auto v : vec) nxt[u][v] = x;
    }
  }
  memset(dp, -1, sizeof dp);
  int ans = 0;
  for (int u = 1; u <= n; u++) {
    for (int v = 1; v <= n; v++) {
      ans = max(ans, f(u, v));
    }
  }
  cout << ans << '\n';
}
```

5.10 SOS DP

```
void AmeDoko() {
  fill(f, f + (1 << N), INT_MAX);
  int n; cin >> n;
  for (int i = 0; i < n; i++) {
    cin >> a[i];
    f[a[i]] = a[i];
  }
  for (int mask = 0; mask < (1 << N);
      mask++) {
    for ( int i = mask; i > 0; i -= (i &
        -i)) {
      int temp = (mask ^ ( i & -i ));
      if ( f[temp] != INT_MAX ) {
        f[mask] = f[temp];
        break;
      }
    }
  }
  for (int i = 0; i < n; i++) {
    if (f[ ~a[i] & ((1 << N) - 1) ] !=
        INT_MAX) cout << f[ ~a[i] & ((1
        << N) - 1) ] << ' ';
    else cout << -1 << ' ';
  }
}
```

6 Graph Theory

6.1 Binary Lifting and LCA

```
const int N = 2e5 + 9, LOG = 20;
vector<int> g[N];
int par[N][LOG], depth[N];
void dfs(int u, int p) {
  par[u][0] = p;
  depth[u] = depth[p] + 1;
  for (int i = 1; i < LOG; i++) {
    par[u][i] = par[par[u][i - 1]][i - 1];
  }
  for (auto v : g[u]) {
    if (v != p) {
      dfs(v, u);
    }
  }
}
int lca(int u, int v) {
  if (depth[u] < depth[v]) {
    swap(u, v);
  }
  int k = depth[u] - depth[v];
  for (int i = 0; i < LOG; i++) {
    if (CHECK(k, i)) u = par[u][i];
  }
  if (u == v) return u;
  for (int i = LOG - 1; i >= 0; i--) {
    if (par[u][i] != par[v][i]) {
      u = par[u][i];
      v = par[v][i];
    }
  }
  return par[u][0];
}
int kth(int u, int k) { // kth parent of
    u
  assert(k >= 0);
  for (int i = 0; i < LOG; i++) {
    if (CHECK(k, i)) u = par[u][i];
  }
  return u;
}
int dist(int u, int v) { // distance from
    u to v
  int l = lca(u, v);
  return (depth[u] - depth[l]) +
      (depth[v] - depth[l]);
}
// kth node from u to v, 0th node is u
int kth(int u, int v, int k) {
  int l = lca(u, v);
  int d = dist(u, v);
  assert(k <= d);
  if (depth[l] + k <= depth[u]) {
    return kth(u, k);
  }
  k -= depth[u] - depth[l];
  return kth(v, depth[v] - depth[l] - k);
}
```

6.2 LCA and Sparse Table on Tree

```
// max and min weights of a path
const int N = 1e5 + 9, LOG = 20, inf =
    1e9; // change here
vector<array<int, 2>> g[N];
int par[N][LOG], tree_mx[N][LOG],
    depth[N];
void dfs(int u, int p, int dis) {
  par[u][0] = p;
  tree_mx[u][0] = dis;
  depth[u] = depth[p] + 1;
  for (int i = 1; i < LOG; i++) {
    par[u][i] = par[par[u][i - 1]][i - 1];
    tree_mx[u][i] = max(tree_mx[u][i -
        1], tree_mx[par[u][i - 1]][i -
        1]);
  }
  for (auto [v, w] : g[u]) {
    if (v != p) {
      dfs(v, u, w);
    }
  }
}
int query_max(int u, int v) { // max
    weight on path u to v
  int l = lca(u, v);
  int d = dist(l, u);
  int ans = 0;
  for (int i = 0; i < LOG; i++) {
    if (CHECK(d, i)) {
      ans = max(ans, tree_mx[u][i]);
      u = par[u][i];
```

```
    }
  }
  d = dist(l, v);
  for (int i = 0; i < LOG; i++) {
    if (CHECK(d, i)) {
      ans = max(ans, tree_mx[v][i]);
      v = par[v][i];
    }
  }
  return ans;
}
```

6.3 Dijkstra

```
vector<int> dijkstra(int s) {
  vector<int> dis(n + 1, inf);
  vector<bool> vis(n + 1, false);
  dis[s] = 0;
  priority_queue<array<int, 2>,
      vector<array<int, 2>>,
      greater<array<int, 2>>> pq;
  pq.push({0, s});
  while (!pq.empty()) {
    auto [d, u] = pq.top(); pq.pop();
    if (vis[u]) continue;
    vis[u] = true;
    for (auto [v, w] : g[u]) {
      if (dis[v] > d + w) {
        dis[v] = d + w;
        pq.push({dis[v], v});
      }
    }
  }
  return dis;
}
```

6.4 Bellman Ford

```
// works for neg edge, can detect neg
    cycle
// Time: O(n^2)
const ll inf = 1e18;
vector<ll> dis(N, inf);
bool bellman_ford(int s) {
  dis[s] = 0;
  bool has_cycle = false;
  for (int i = 1; i <= n; i++) {
    for (int u = 1; u <= n; u++) {
      for (auto [v, w] : g[u]) {
        if (dis[v] > dis[u] + w) {
          if (i == n) has_cycle = true;
          dis[v] = dis[u] + w;
        }
      }
    }
  }
  return has_cycle;
}
```

6.5 Floyd Warshall

```
// dis[i][j] = min distance to reach i to
    j, works for neg edge (no neg cycle)
// Time: O(n^3)
vector<int> construct_path(int u, int v)
    { // O(n)
  if (nxt[u][v] == -1) return {};
  vector<int> path = { u };
  while (u != v) {
    u = nxt[u][v];
    path.push_back(u);
  }
  return path;
}
void floyd_warshall() {
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= n; j++) {
      if (i == j) dis[i][j] = 0;
      else if (g[i][j] == 0) dis[i][j] =
          inf;
      else dis[i][j] = g[i][j];
    }
  }
  for (int k = 1; k <= n; ++k) {
    for (int i = 1; i <= n; ++i) {
      for (int j = 1; j <= n; ++j) {
        if (dis[i][k] < inf and
            dis[k][j] < inf)
          dis[i][j] = min(dis[i][j],
              dis[i][k] + dis[k][j]);
          nxt[i][j] = nxt[i][k];
      }
    }
  }
}
int32_t main() {
  int q; cin >> n >> m >> q;
  memset(nxt, -1, sizeof nxt);
  while (m--) {
    int u, v, w; cin >> u >> v >> w;
    g[u][v] = (g[u][v] != 0 ?
        min(g[u][v], w) : w);
    g[v][u] = (g[v][u] != 0 ?
        min(g[v][u], w) : w);
    nxt[u][v] = v;
    nxt[v][u] = u;
  }
  floyd_warshall();
  while (q--) {
    int u, v; cin >> u >> v;
    cout << (dis[u][v] == inf ? -1 :
        dis[u][v]) << '\n';
  }
  return 0;
}
```

6.6 Strongly Connected Components

```
// Time: O(n + m)
const int N = 1e5 + 9;
vector<int> g[N], gT[N], G[N];
vector<bool> vis(N, false);
vector<vector<int>> components;
vector<int> order;
int n, roots[N], sz[N];
void dfs(int u) {
  vis[u] = true;
  for (auto v : g[u]) {
    if (!vis[v]) dfs(v);
  }
  order.push_back(u);
}
void dfs2(int u, vector<int> &component) {
  vis[u] = true;
  component.push_back(u);
  for (auto v : gT[u]) {
    if (!vis[v]) dfs2(v, component);
  }
}
void scc() {
  // get order sorted by end time
  order.clear();
  for (int u = 1; u <= n; u++) {
    if (!vis[u]) dfs(u);
  }
  reverse(order.begin(), order.end());
  // transpose the graph
  for (int u = 1; u <= n; u++) {
    for (auto v : g[u]) {
      gT[v].push_back(u);
    }
  }
  // get all components
  components.clear();
  for (int i = 1; i <= n; i++) vis[i] =
      false;
  for (auto u : order) {
    if (!vis[u]) {
      vector<int> component;
      dfs2(u, component);
      sort(component.begin(),
          component.end());
      components.push_back(component);
      for (auto v : component) {
        roots[v] = component.front();
        sz[v] = component.size();
      }
    }
  }
  // add edges to condensation graph
  for (int u = 1; u <= n; u++) {
    for (auto v : g[u]) {
      if (roots[u] != roots[v]) {
        G[roots[u]].push_back(roots[v]);
      }
    }
  }
}
// when you need to use condensed graph,
    use it carefully (Specially g->G,
    i->roots[i])
```

6.7 Articulation Points

```
int disc[N], low[N], timer, n;
vector<bool> vis(N, false), is_ap(N,
    false);
void ap_dfs(int u, int p) {
  disc[u] = low[u] = ++timer;
  vis[u] = true;
  int children_cnt = 0;
  for (auto v : g[u]) {
    if (v == p) continue;
    if (vis[v]) low[u] = min(low[u],
        disc[v]);
    else {
      ap_dfs(v, u);
      low[u] = min(low[u], low[v]);
      if (disc[u] <= low[v] and p != -1)
          is_ap[u] = true;
      children_cnt++;
    }
  }
  if (p == -1 and children_cnt > 1)
      is_ap[u] = true;
}
void find_articulation_points() {
  for (int u = 1; u <= n; u++) {
    if (!vis[u]) {
      timer = 0;
      ap_dfs(u, -1);
    }
  }
}
```

6.8 Find Bridges

```
map<pair<int, int>, int> bridges;
void bridges_dfs(int u, int p) { // find
    bridges
  disc[u] = low[u] = ++timer;
  vis[u] = true;
  for (auto v : g[u]) {
    if (v == p) continue;
    if (vis[v]) low[u] = min(low[u],
        disc[v]);
    else {
      bridges_dfs(v, u);
      low[u] = min(low[u], low[v]);
      if (disc[u] < low[v]) {
        bridges[(make_pair(min(u, v),
            max(u, v)))]++;
      }
    }
  }
}
void find_bridges() {
  for (int u = 1; u <= n; u++) {
    if (!vis[u]) {
      timer = 0;
      bridges_dfs(u, -1);
    }
  }
}
```

6.9 DSU on Tree

```
  // Problem: CSES/task/2081 (Number of
    distinct path of len k_1...k_2)
// Complexity: O(n(logn)^2)
int par[N], ans; // par[i] = i initially
deque<int> suff[N];
int find(int i) {
  return (i == par[i] ? i : par[i] =
      find(par[i]));
}
int get_sum(int node, int l, int r) {
  if (l > r) return 0;
  return suff[node][l] - (r + 1 <
      suff[node].size() ? suff[node][r +
      1] : 0);
}
void merge(int u, int v) {
  if ((u = find(u)) == (v = find(v)))
      return;
```

```
  bool swapped = false;
  if (suff[u].size() >= suff[v].size()) {
    swap(u, v); swapped = true;
  }
  for (int i = 0; i < suff[u].size();
      i++) {
    int x = suff[u][i] - (i + 1 <
        suff[u].size() ? suff[u][i + 1]
        : 0);
    int need_mn = max(0ll, k1 - i - 1);
    int tmp = suff[v].size() - 1;
    int need_mx = min(tmp, k2 - i - 1);
    ans += get_sum(v, need_mn, need_mx)
        * x;
  }
  // merging two suffix sum
  if (swapped) { // normal way (child
      size choto)
    suff[v][0] += suff[u][0]; int idx = 1;
    for (auto x : suff[u]) {
      if (idx < suff[v].size())
          suff[v][idx] += x;
      else suff[v].push_back(x);
      idx++;
    }
  }
  else { // child size boro
    int idx = 0; bool first = true;
    for (auto x : suff[u]) {
      if (first) first = false; continue;
      if (idx < suff[v].size())
          suff[v][idx] += x;
      else suff[v].push_back(x);
      idx++;
    }
    suff[v].push_front(1 +
        suff[v].front());
  }
  par[u] = v;
}
void dfs(int u, int p) {
  suff[u].push_back(1); // initially 0
      len er cnt 1
  for (auto v : g[u]) {
    if (v != p) {
      dfs(v, u);
      merge(u, v);
    }
  }
}
```

6.10 Heavy-Light Decomposition

```
// Per Query Complexity: O(logn^2)
// Path and subtree updates and queries.
const int N = 2e5 + 9, LOG = 20, inf =
    1e9; // change here
vector<int> g[N];
int par[N][LOG], depth[N], sz[N];
int disc[N], finish[N], timer, head[N];
int n;
void dfs(int u, int p = 0) {
  par[u][0] = p;
  depth[u] = depth[p] + 1;
  sz[u] = 1;
  for (int i = 1; i < LOG; i++) {
    par[u][i] = par[par[u][i - 1]][i - 1];
  }
  if (p) g[u].erase(find(g[u].begin(),
      g[u].end(), p));
  for (auto &v : g[u]) {
    if (v != p) {
      dfs(v, u);
      sz[u] += sz[v];
      if (sz[v] > sz[g[u][0]]) swap(v,
          g[u][0]);
    }
  }
}
void dfs_hld(int u) {
  disc[u] = ++timer;
  for (auto v : g[u]) {
    head[v] = (v == g[u][0] ? head[u] :
        v);
    dfs_hld(v);
  }
  finish[u] = timer;
}
int kth(int u, int k) {
  assert(k >= 0);
  for (int i = 0; i < LOG; i++) {
    if (CHECK(k, i)) u = par[u][i];
  }
  return u;
}
int query_up(int u, int v) {
  int ans = -inf; // change here
  while (head[u] != head[v]) {
    ans = max(ans, st.query(1, 1, n,
        disc[head[u]], disc[u])); //
        change here
    u = par[head[u]][0];
  }
  ans = max(ans, st.query(1, 1, n,
      disc[v], disc[u])); // change here
  return ans;
}
int query(int u, int v) {
  int lc = lca(u, v);
  int ans = query_up(u, lc);
  if (v != lc) {
    ans = max(ans, query_up(v, kth(v,
        depth[v] - depth[lc] - 1))); //
        change here
  }
  return ans;
}
void solve() {
  cin >> n;
  for (int i = 2; i <= n; i++) {
    int u, v; cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u);
  }
  dfs(1);
  head[1] = 1;
  dfs_hld(1);
  st.build(1, 1, n);
}
```

6.11 Inverse Graph

```
// Problem: CF/0-1 MST
  // A Complete Graph. m edges have
     weight 1, rest have 0. MST?
const int N = 200000 + 9;
set<int> g[N], non_vis;
void dfs(int u) {
  non_vis.erase(u);
  for (int v = 1; v <= n; v++) {
    if (v == u) continue;
    auto it = non_vis.lower_bound(v);
    if (it == non_vis.end()) break;
    v = *it;
    if (g[u].find(v) != g[u].end())
        continue;
    dfs(v);
  }
}
int32_t main() {
  int m; cin >> n >> m;
  while (m--) {
    int u, v; cin >> u >> v;
    g[u].insert(v);
    g[v].insert(u);
  }
  for (int i = 1; i <= n; i++)
     non_vis.insert(i);
  int component_cnt = 0;
  for (int i = 1; i <= n; i++) {
    if (non_vis.find(i) !=
        non_vis.end()) {
      component_cnt++;
      dfs(i);
    }
  }
  cout << component_cnt - 1 << '\n';
}
```

6.12 [Problem] Rooted Tree - Hackerrank

Given a rooted tree, Upd: Add $d * k$ on v(subtree nodes of u), where k will be given and d is distance of v from u. Query: Node Value/Subtree Sum (using ETT) or Path Value Sum (using HLD). **Idea:** d_i = dis from root to i. Then, ans $= k \cdot \sum_{v \in \text{subtree}(u)} d_v$. It's hard to find d_v for each u, rather we can find d_v = dis of v from root. Although some extra value added on each v of subtree u because of d_v from root (not u), we can easily subtract them ($depth[u] * k$).

7 String

7.1 Hashing

```
const int N = 1e6 + 9; // change here
const int MOD1 = 127657753, MOD2 =
    987654319;
const int p1 = 137, p2 = 277; // change
    here
int ip1, ip2;
pair<int, int> pw[N], ipw[N];
void prec() {
  pw[0] = {1, 1};
  for (int i = 1; i < N; i++) {
    pw[i].first = 1ll * pw[i - 1].first
        * p1 % MOD1;
    pw[i].second = 1ll * pw[i -
        1].second * p2 % MOD2;
  }
  ip1 = power(p1, MOD1 - 2, MOD1);
  ip2 = power(p2, MOD2 - 2, MOD2);
  ipw[0] = {1, 1};
  for (int i = 1; i < N; i++) {
    ipw[i].first = 1ll * ipw[i -
        1].first * ip1 % MOD1;
    ipw[i].second = 1ll * ipw[i -
        1].second * ip2 % MOD2;
  }
}
struct Hashing {
  int n;
  string s;
  vector<pair<int, int>> hash_val;
  vector<pair<int, int>> rev_hash_val;
  Hashing() {}
  Hashing(string _s) {
    s = _s;
    n = s.size();
    hash_val.emplace_back(0, 0);
    for (int i = 0; i < n; i++) {
      pair<int, int> p;
      p.first = (hash_val[i].first + 1ll
          * s[i] * pw[i].first % MOD1) %
          MOD1;
      p.second = (hash_val[i].second +
          1ll * s[i] * pw[i].second %
          MOD2) % MOD2;
      hash_val.push_back(p);
    }
    rev_hash_val.emplace_back(0, 0);
    for (int i = 0, j = n - 1; i < n;
        i++, j--) {
      pair<int, int> p;
      p.first = (rev_hash_val[i].first +
          1ll * s[i] * pw[j].first %
          MOD1) % MOD1;
      p.second = (rev_hash_val[i].second
          + 1ll * s[i] * pw[j].second %
          MOD2) % MOD2;
      rev_hash_val.push_back(p);
    }
  }
  pair<int, int> get_hash(int l, int r)
      { // 1 indexed
    pair<int, int> ans;
    ans.first = (hash_val[r].first -
        hash_val[l - 1].first + MOD1) *
        1ll * ipw[l - 1].first % MOD1;
    ans.second = (hash_val[r].second -
        hash_val[l - 1].second + MOD2) *
        1ll * ipw[l - 1].second % MOD2;
    return ans;
  }
  pair<int, int> rev_hash(int l, int r)
      { // 1 indexed
    pair<int, int> ans;
```

```
    ans.first = (rev_hash_val[r].first -
        rev_hash_val[l - 1].first +
        MOD1) * 1ll * ipw[n - r].first %
        MOD1;
    ans.second = (rev_hash_val[r].second
        - rev_hash_val[l - 1].second +
        MOD2) * 1ll * ipw[n - r].second
        % MOD2;
    return ans;
  }
  pair<int, int> get_hash() { // 1
      indexed
    return get_hash(1, n);
  }
  bool is_palindrome(int l, int r) {
    return get_hash(l, r) == rev_hash(l,
        r);
  }
};
```

7.2 Hashing with Updates

```
using T = array<int, 2>;
const T MOD = {127657753, 987654319};
const T p = {137, 277}; // change here
T operator + (T a, int x) {return {(a[0]
    + x) % MOD[0], (a[1] + x) % MOD[1]};}
T operator - (T a, int x) {return {(a[0]
    - x + MOD[0]) % MOD[0], (a[1] - x +
    MOD[1]) % MOD[1]};}
T operator * (T a, int x) {return
    {(int)((long long) a[0] * x %
    MOD[0]), (int)((long long) a[1] * x
    % MOD[1])};}
T operator + (T a, T x) {return {(a[0] +
    x[0]) % MOD[0], (a[1] + x[1]) %
    MOD[1]};}
T operator - (T a, T x) {return {(a[0] -
    x[0] + MOD[0]) % MOD[0], (a[1] -
    x[1] + MOD[1]) % MOD[1]};}
T operator * (T a, T x) {return
    {(int)((long long) a[0] * x[0] %
    MOD[0]), (int)((long long) a[1] *
    x[1] % MOD[1])};}
ostream& operator << (ostream& os, T
    hash) {return os << "(" << hash[0]
    << ", " << hash[1] << ")";}
T pw[N], ipw[N];
void prec() {
  pw[0] =  {1, 1};
  for (int i = 1; i < N; i++) {
    pw[i] = pw[i - 1] * p;
  }
  ipw[0] =  {1, 1};
  T ip = {power(p[0], MOD[0] - 2,
      MOD[0]), power(p[1], MOD[1] - 2,
      MOD[1])};
  for (int i = 1; i < N; i++) {
    ipw[i] = ipw[i - 1] * ip;
  }
}
struct Hashing {
  int n;
  string s; // 1 - indexed
  vector<array<T, 2>> t; // (normal, rev)
      hash
  array<T, 2> merge(array<T, 2> l,
      array<T, 2> r) {
    l[0] = l[0] + r[0];
    l[1] = l[1] + r[1];
    return l;
  }
  void build(int node, int b, int e) {
    if (b == e) {
      t[node][0] = pw[b] * s[b];
      t[node][1] = pw[n - b + 1] * s[b];
      return;
    }
    int mid = (b + e) >> 1, l = node <<
        1, r = l | 1;
    build(l, b, mid);
    build(r, mid + 1, e);
    t[node] = merge(t[l], t[r]);
  }
  void upd(int node, int b, int e, int
      i, char x) {
    if (b > i || e < i) return;
    if (b == e && b == i) {
      t[node][0] = pw[b] * x;
      t[node][1] = pw[n - b + 1] * x;
      return;
    }
    int mid = (b + e) >> 1, l = node <<
        1, r = l | 1;
    upd(l, b, mid, i, x);
    upd(r, mid + 1, e, i, x);
    t[node] = merge(t[l], t[r]);
  }
  array<T, 2> query(int node, int b, int
      e, int i, int j) {
    if (b > j || e < i) return {T({0,
        0}), T({0, 0})};
    if (b >= i && e <= j) return t[node];
    int mid = (b + e) >> 1, l = node <<
        1, r = l | 1;
    return merge(query(l, b, mid, i, j),
        query(r, mid + 1, e, i, j));
  }
  Hashing() {}
  Hashing(string _s) {
    n = _s.size();
    s = "." + _s;
    t.resize(4 * n + 1);
    build(1, 1, n);
  }
  void upd(int i, char c) {
    upd(1, 1, n, i, c);
    s[i] = c;
  }
  T get_hash(int l, int r) { // 1 -
      indexed
    return query(1, 1, n, l, r)[0] *
        ipw[l - 1];
  }
  T rev_hash(int l, int r) { // 1 -
      indexed
    return query(1, 1, n, l, r)[1] *
        ipw[n - r];
  }
  T get_hash() {
    return get_hash(1, n);
  }
  bool is_palindrome(int l, int r) {
    return get_hash(l, r) == rev_hash(l,
        r);
  }
};
```

7.3 Hashing with Upd and Deletes

```
// update or delete a char in the string
    or check whether a range [l,r] is a
    palindrome or not (Palindromic Query I
    - Toph)
#define int  long long
const int N = 1e5 + 9;
int en;
struct ST {
  pair<int, int> tree[4 * (N + N)];
  void build(int n, int b, int e) {
    if (b == e) {
      tree[n].first = b;
      tree[n].second = 1;
      return;
    }
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    build(l, b, mid);
    build(r, mid + 1, e);
    tree[n].second = tree[l].second +
        tree[r].second;
  }
  void upd(int n, int b, int e, int i,
      int x1, int x2) {
    if (b > i || e < i) return;
    if (b == e && b == i) {
      tree[n].first = x1;
      tree[n].second = x2;
      return;
    }
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    upd(l, b, mid, i, x1, x2);
    upd(r, mid + 1, e, i, x1, x2);
    tree[n].second = tree[l].second +
        tree[r].second;
  }
  pair<int, int> query(int n, int b, int
      e, int x) {
    if (b > e) return { -1, -1};
    if (tree[n].second < x) return
        {tree[n].second, -1};
    if (b == e) return tree[n];
    int mid = (b + e) >> 1, l = n << 1,
        r = l + 1;
    pair<int, int> L = query(l, b, mid,
        x);
    if (L.second != -1) return L;
    pair<int, int> R = query(r, mid + 1,
        e, x - L.first);
    return R;
  }
} st, st2;
using T = array<int, 2>;
const T MOD = {127657753, 987654319};
const T p = {137, 277};
// add operators overloading of T (from
    only upd) + prec()
int get(int i, int n) {
  return n - i + 1;
}
struct Hashing {
  int n; string s;
  vector<T> tree, lazy;
  void push(int node, int b, int e) {
    if (lazy[node][0] == 1) return;
    tree[node] = tree[node] * lazy[node];
    if (b != e) {
      int l = node << 1, r = l + 1;
      lazy[l] = lazy[l] * lazy[node];
      lazy[r] = lazy[r] * lazy[node];
    }
    lazy[node] = T{1, 1};
  }
  void build(int node, int b, int e) {
    lazy[node] = T{1, 1};
    if (b == e) {
      tree[node] = pw[b] * s[b];
      return;
    }
    int mid = (b + e) >> 1, l = node <<
        1, r = l | 1;
    build(l, b, mid);
    build(r, mid + 1, e);
    tree[node] = tree[l] + tree[r];
  }
  void upd(int node, int b, int e, int
      i, T x) {
    push(node, b, e);
    if (b > i || e < i) return;
    if (b == e && b == i) {
      tree[node] = x;
      return;
    }
    int mid = (b + e) >> 1, l = node <<
        1, r = l + 1;
    upd(l, b, mid, i, x);
    upd(r, mid + 1, e, i, x);
    tree[node] = tree[l] + tree[r];
  }
  void del(int node, int b, int e, int
      i, int j) {
    push(node, b, e);
    if (b > j || e < i) return;
    if (b >= i && e <= j) {
      lazy[node] = lazy[node] * ipw[1];
      push(node, b, e);
      return;
    }
    int mid = (b + e) >> 1, l = node <<
        1, r = l + 1;
    del(l, b, mid, i, j);
    del(r, mid + 1, e, i, j);
    tree[node] = tree[l] + tree[r];
  }
  T query(int node, int b, int e, int i,
      int j) {
    push(node, b, e);
    if (b > j || e < i) return {0, 0};
```

```
    if (b >= i && e <= j) return
        tree[node];
    int mid = (b + e) >> 1, l = node <<
        1, r = l + 1;
    T L = query(l, b, mid, i, j);
    T R = query(r, mid + 1, e, i, j);
    return L + R;
  }
  Hashing() {}
  Hashing(string _s) {
    s = _s;
    n = s.size();
    s = '.' + s;
    tree.resize(4 * n + 1);
    lazy.resize(4 * n + 1);
    build(1, 1, n);
  }
  void upd(int i, char c, int cur) {
    T x = pw[i] * c;
    if (cur == 1) i = st.query(1, 1, en,
        i).first;
    else i = st2.query(1, 1, en, i).first;
    upd(1, 1, n, i, x);
  }
  void del(int i, int cur) {
    int orgi = i;
    T x = pw[i] * 0ll;
    if (cur == 1) i = st.query(1, 1, en,
        i).first;
    else i = st2.query(1, 1, en, i).first;
    upd(1, 1, n, i, x);
    del(1, 1, n, i + 1, n);
    if (cur == 1) st.upd(1, 1, en, i, i,
        0);
    else st2.upd(1, 1, en, i, i, 0);
  }
  T get_hash(int l, int r, int cur) { // 1
      - indexed
    int ll = st.query(1, 1, en, l).first;
    int rr = st.query(1, 1, en, r).first;
    if (cur == 2) {
      ll = st2.query(1, 1, en, l).first;
      rr = st2.query(1, 1, en, r).first;
    }
    return query(1, 1, n, ll, rr) *
        ipw[l - 1];
  }
};
int32_t main() {
  prec(); // must include
  string s; cin >> s;
  int n = s.size();
  int q; cin >> q;
  string t = s;
  reverse(t.begin(), t.end());
  Hashing hs(s), hs2(t);
  en = n + q + 5;
  st.build(1, 1, en);
  st2.build(1, 1, en);
  while (q--) {
    char c; cin >> c;
    if (c == 'C') {
      int l, r; cin >> l >> r;
      int l2 = get(l, n);
      int r2 = get(r, n);
      if (hs.get_hash(l, r, 1) ==
          hs2.get_hash(r2, l2, 2)) cout
          << "Yes!\n";
      else cout << "No!\n";
    }
    else if (c == 'U') {
      int i; char x; cin >> i >> x;
      int i2 = get(i, n);
      hs.upd(i, x, 1);
      hs2.upd(i2, x, 2);
    }
    else {
      int i; cin >> i;
      int i2 = get(i, n);
      hs.del(i, 1);
      hs2.del(i2, 2);
      --n;
    }
  }
}
```

7.4 Hashing on Tree

```
// Given a tree, Check whether it is
    symmetrical or not. Problem - CF G.
    Symmetree
// The value for each node is it's
    subtree size and position is the
    level (ordered). But the order of
    childs doesn't matter (unordered)
const int N = 2e5 + 9;
vector<int> g[N];
vector<array<int, 3>> hassh[N]; // hash1,
    hash2, node
int n, sz[N];
const int MOD1 = 1e9 + 9, MOD2 = 1e9 + 21;
const int p1 = 1e5 + 19, p2 = 1e5 + 43;
void dfs2(int u, int p, int lvl) {
  array<int, 3> my_hash;
  my_hash[0] = 1ll * sz[u] *
      pw[lvl].first % MOD1;
  my_hash[1] = 1ll * sz[u] *
      pw[lvl].second % MOD2;
  my_hash[2] = u;
  bool leaf = true;
  for (auto v : g[u]) {
    if (v != p) {
      dfs2(v, u, lvl + 1);
      leaf = false;
    }
  }
  if (!leaf) {
    int sum1 = 1, sum2 = 1;
    for (auto here : hassh[u]) {
      auto [x, y, _] = here;
      sum1 = (sum1 * x) % MOD1;
      sum2 = (sum2 * y) % MOD2;
    }
    my_hash[0] = power(my_hash[0], sum1,
        MOD1);
    my_hash[1] = power(my_hash[1], sum2,
        MOD2);
  }
  hassh[p].push_back(my_hash);
}
bool ok(int u) {
  map<pair<int, int>, int> mp;
  for (auto [x, y, who] : hassh[u]) {
    mp[{x, y}]++;
  }
  int odd = 0;
  pair<int, int> val;
  for (auto [here, cnt] : mp) {
    odd += cnt & 1;
    if (cnt & 1) val = here;
  }
  if (odd == 0) return true;
  if (odd > 1) return false;
  int node;
  for (auto [x, y, who] : hassh[u]) {
    pair<int, int> here = {x, y};
    if (here == val) node = who;
  }
  return ok(node);
}
void solve() {
  cin >> n; clr(n);
  for (int i = 2; i <= n; i++) {
    int u, v; cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u);
  }
  dfs(1, 0); // calc. subtree size
  dfs2(1, 0, 1);
  if (ok(0)) cout << "YES\n";
  else cout << "NO\n";
}
```

7.5 Compare 2 strings Lexicographically

```
// Time: O(logn)
string s;
Hashing hs;
// return 0 if both equal
// return 1 if first substring greater
// return -1 if second substring greater
// here lcp() provides the len of longest
    common prefix
int compare(int i, int j, int x, int y) {
  int common_prefix = lcp(i, j, x, y);
  int len1 = j - i + 1, len2 = y - x + 1;
  if (common_prefix == len1 and len1 ==
      len2) return 0;
  else if (common_prefix == len1) return
      -1;
  else if (common_prefix == len2) return
      1;
  else return (s[i + common_prefix - 1]
      < s[x + common_prefix - 1] ? -1 :
      1);
}
```

7.6 KMP

```
vector<int> build_lps(string &pat) {
  int n = pat.size();
  vector<int> lps(n, 0);
  for (int i = 1; i < n; i++) {
    int j = lps[i - 1];
    while (j > 0 and pat[i] != pat[j]) {
      j = lps[j - 1];
    }
    if (pat[i] == pat[j]) j++;
    lps[i] = j;
  }
  return lps;
}
int kmp(string &txt, string &pat) {
  string s = pat + '#' + txt;
  vector<int> lps = build_lps(s);
  int ans = 0;
  for (auto x : lps) {
    if (x == pat.size()) ans++;
  }
  return ans;
}
int kmp(string &txt, string &pat) {
  vector<int> lps = build_lps(pat);
  int n = txt.size(), m = pat.size();
  int ans = 0;
  int j = 0;
  for (int i = 0; i < n; i++) {
    while (j > 0 and txt[i] != pat[j]) {
      j = lps[j - 1];
    }
    if (txt[i] == pat[j]) j++;
    if (j == m) {
      ans++;
      j = lps[j - 1];
    }
  }
  return ans;
}
```

7.7 KMP Automata

```
// like DFA. if string is "abcdeabg",
    aut[7]['c'] = 3. Means 7th index e
    'c' bosaile LPS koto, aut[7]['g'] = 8
void compute_automaton(string s,
    vector<vector<int>>& aut) {
  s += '#';
  int n = s.size();
  vector<int> pi = build_lps(s);
  aut.assign(n, vector<int>(26));
  for (int i = 0; i < n; i++) {
    for (int c = 0; c < 26; c++) {
      if (i > 0 && 'a' + c != s[i])
        aut[i][c] = aut[pi[i - 1]][c];
      else
        aut[i][c] = i + ('a' + c == s[i]);
    }
  }
}
```

7.8 Prefix Occurance Count

```
// Count the number of occurances of each
    prefix
vector<int> ans(n + 1);
for (int i = 0; i < n; i++) ans[lps[i]]++;
for (int i = n - 1; i > 0; i--)
    ans[lps[i - 1]] += ans[i];
for (int i = 0; i <= n; i++) ans[i]++;
```

7.9 Number of palindormic substring in L to R using Wavelet Tree

```
// Problem - Kattis palindromes
ll f(int x) {
  return (1ll * x * (x + 1)) / 2;
}
ll f(int l, int r) {
  if (l > r) return 0;
  return f(r) - f(l - 1);
}
bool ok(int l, int r) {
  return hash_s.is_palindrome(l, r);
}
int32_t main() {
  cin >> s;
  n = s.size();
  hash_s = Hashing(s);
  for (int i = 1; i <= n; i++) {
    int l = 0, r = min(n - i, i - 1),
        cnt = 1;
    while (l <= r) {
      int mid = (l + r) >> 1;
      if (ok(i - mid, i + mid)) {
        cnt = mid;
        l = mid + 1;
      }
      else r = mid - 1;
    }
    pi1[i] = cnt + 1;
    pi1_left[i] = pi1[i] - i;
    pi1_right[i] = i + pi1[i];
  }
  for (int i = 2; i <= n; i++) {
    if (s[i - 1] == s[i - 2]) {
      int l = 0, r = min(n - i, i - 1),
          cnt = 2;
      while (l <= r) {
        int mid = (l + r) >> 1;
        if (ok(i - 1 - mid, i + mid)) {
          cnt = mid;
          l = mid + 1;
        }
        else r = mid - 1;
      }
      pi2[i] = cnt + 1;
    }
    else pi2[i] = 0;

    pi2_left[i] = pi2[i] - i;
    pi2_right[i] = i + pi2[i];
  }
  // wavelet trees (odd_len_left,
      odd_len_right, even_len_left,
      even_len_right)
  t1.init(pi1_left + 1, pi1_left + n +
      1, -N, N);
  t2.init(pi1_right + 1, pi1_right + n +
      1, -N, N);
  t3.init(pi2_left + 1, pi2_left + n +
      1, -N, N);
  t4.init(pi2_right + 1, pi2_right + n +
      1, -N, N);

  int q; cin >> q;
  while (q--) {
    int l, r; cin >> l >> r;
    // define k, find cnt > k and
        summation whose are <= k;
    int m = (l + r) / 2;
    int k = 1 - l;
    ll ans = f(l, m);
    ans += t1.sum(l, m, k);
    int cnt = t1.GT(l, m, k);
    ans += 1ll * k * cnt;
    k = 1 + r;
    ans += -f(m + 1, r);
    ans += t2.sum(m + 1, r, k);
    cnt = t2.GT(m + 1, r, k);
    ans += 1ll * k * cnt;
    if (l + 1 <= m) { // a bit different
        than others
      k = -l;
      ans += f(l + 1, m);
      ans += t3.sum(l + 1, m, k);
      cnt = t3.GT(l + 1, m, k);
      ans += 1ll * k * cnt;
    }
    k = 1 + r;
    ans += -f(m + 1, r);
    ans += t4.sum(m + 1, r, k);
    cnt = t4.GT(m + 1, r, k);
    ans += 1ll * k * cnt;
    cout << ans << '\n';
  }
}
```

It is easier to explain by considering only palindromes centered at indicies (so, odd length), the idea is the same anyway. For each index i, r_i will be the longest radius of a palindrome centered there (in other words, the amount of palindromes centered at index i). Directly from manacher, this takes $\mathcal{O}(n)$ to calculate.

For a query $[l, r]$, we first compute $m = \frac{l+r}{2}$. Now we want to calculate

$$\sum_{i=l}^{m} \min(i - l + 1, r_i) + \sum_{i=m+1}^{r} \min(r - i + 1, r_i)$$

$$\sum_{i=l}^{m} \min(i - l + 1, r_i) = \sum_{i=l}^{m} i + \min(1 - l, r_i - i).$$

The sum over i can be found in constant time. As for the other term, if we create some array $r'_i = r_i - i$ during the preprocessing, then the queries are asking for some over range of $\min(C, r'_i)$ where C is constant. You can solve this in $\mathcal{O}(\log n)$ per query using wavelet tree.

7.10 Trie

```
const int N = 10; // change here
const char base_char = '0';  // change
    here
struct TrieNode {
  int cnt;
  TrieNode * nxt[N];
  TrieNode() {
    cnt = 0;
    for (int i = 0; i < N; i++) nxt[i] =
        NULL;
  }
} *root;
void insert(const string &s) {
  TrieNode *cur = root;
  int n = (int)s.size();
  for (int i = 0; i < n; i++) {
    int idx = s[i] - base_char;
    if (cur -> nxt[idx] == NULL) cur ->
        nxt[idx] = new TrieNode();
    cur = cur -> nxt[idx];
    cur -> cnt++;
  }
}
void rem(TrieNode *cur, string &s, int
    pos) { // free :: De Alloactes Memory
  if (pos == s.size()) return;
  int idx = s[pos] - base_char;
  rem(cur -> nxt[idx], s, pos + 1);
  cur -> nxt[idx] -> cnt--;
  if (cur -> nxt[idx] -> cnt == 0) {
    free(cur -> nxt[idx]);
    cur -> nxt[idx] = NULL;
  }
}
int query(const string &s) { // "s" is a
    prefix of some element or not
  int n = (int)s.size();
  TrieNode *cur = root;
  for (int i = 0; i < n; i++) {
    int idx = s[i] - base_char;
    if (cur -> nxt[idx] == NULL) return 0;
    cur = cur -> nxt[idx];
  }
  return cur -> cnt;
}
void del(TrieNode *cur) {
  for (int i = 0; i < N; i++) if (cur ->
      nxt[i]) del(cur -> nxt[i]);
  delete(cur);
}
int32_t main() {
  root = new TrieNode(); // init new trie
  del(root); // clear trie
}
```

7.11 Manachar

```
vector<int> manacher_odd(string s) {
  int n = s.size();
  s = "$" + s + "^";
  vector<int> p(n + 2);
  int l = 0, r = 1;
  for (int i = 1; i <= n; i++) {
    p[i] = min(r - i, p[l + (r - i)]);
    while (s[i - p[i]] == s[i + p[i]]) {
      p[i]++;
    }
    if (i + p[i] > r) {
      l = i - p[i], r = i + p[i];
    }
  }
  return vector<int>(begin(p) + 1,
      end(p) - 1);
}
vector<int> manacher(string s) {
  string t;
  for (auto c : s) {
    t += string("#") + c;
  }
  cout << t << '\n';
  auto res = manacher_odd(t + "#");
  return vector<int>(begin(res) + 1,
      end(res) - 1);
}
```

7.12 Suffix Array

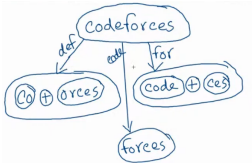
```
const int N = 3e5 + 9, LG = 18; // change
    here
// credit: youknowwho
void induced_sort(const vector<int>
    &vec, int val_range, vector<int>
    &SA, const vector<bool> &sl, const
    vector<int> &lms_idx) {
  vector<int> l(val_range, 0),
      r(val_range, 0);
  for (int c : vec) {
    if (c + 1 < val_range) ++l[c + 1];
    ++r[c];
  }
  partial_sum(l.begin(), l.end(),
      l.begin());
  partial_sum(r.begin(), r.end(),
      r.begin());
  fill(SA.begin(), SA.end(), -1);
  for (int i = lms_idx.size() - 1; i >=
      0; --i)
    SA[--r[vec[lms_idx[i]]]] = lms_idx[i];
  for (int i : SA)
    if (i >= 1 && sl[i - 1]) {
      SA[l[vec[i - 1]]++] = i - 1;
    }
  fill(r.begin(), r.end(), 0);
  for (int c : vec)
    ++r[c];
  partial_sum(r.begin(), r.end(),
      r.begin());
  for (int k = SA.size() - 1, i = SA[k];
      k >= 1; --k, i = SA[k])
    if (i >= 1 && !sl[i - 1]) {
      SA[--r[vec[i - 1]]] = i - 1;
    }
}
vector<int> SA_IS(const vector<int>
    &vec, int val_range) {
  const int n = vec.size();
  vector<int> SA(n), lms_idx;
  vector<bool> sl(n);
  sl[n - 1] = false;
  for (int i = n - 2; i >= 0; --i) {
    sl[i] = (vec[i] > vec[i + 1] ||
        (vec[i] == vec[i + 1] && sl[i +
        1]));
    if (sl[i] && !sl[i + 1])
      lms_idx.push_back(i + 1);
  }
  reverse(lms_idx.begin(), lms_idx.end());
```

```
  induced_sort(vec, val_range, SA, sl,
      lms_idx);
  vector<int>
      new_lms_idx(lms_idx.size()),
      lms_vec(lms_idx.size());
  for (int i = 0, k = 0; i < n; ++i)
    if (!sl[SA[i]] && SA[i] >= 1 &&
        sl[SA[i] - 1]) {
      new_lms_idx[k++] = SA[i];
    }
  int cur = 0;
  SA[n - 1] = cur;
  for (size_t k = 1; k <
      new_lms_idx.size(); ++k) {
    int i = new_lms_idx[k - 1], j =
        new_lms_idx[k];
    if (vec[i] != vec[j]) {
      SA[j] = ++cur;
      continue;
    }
    bool flag = false;
    for (int a = i + 1, b = j + 1;; ++a,
        ++b) {
      if (vec[a] != vec[b]) {
        flag = true;
        break;
      }
      if ((!sl[a] && sl[a - 1]) ||
          (!sl[b] && sl[b - 1])) {
        flag = !((!sl[a] && sl[a - 1])
            && (!sl[b] && sl[b - 1]));
        break;
      }
    }
    SA[j] = (flag ? ++cur : cur);
  }
  for (size_t i = 0; i < lms_idx.size();
      ++i)
    lms_vec[i] = SA[lms_idx[i]];
  if (cur + 1 < (int)lms_idx.size()) {
    auto lms_SA = SA_IS(lms_vec, cur + 1);
    for (size_t i = 0; i <
        lms_idx.size(); ++i) {
      new_lms_idx[i] = lms_idx[lms_SA[i]];
    }
  }
  induced_sort(vec, val_range, SA, sl,
      new_lms_idx);
  return SA;
}
vector<int> suffix_array(const string
    &s, const int LIM = 128) {
  vector<int> vec(s.size() + 1);
  copy(begin(s), end(s), begin(vec));
  vec.back() = '!';
  auto ret = SA_IS(vec, LIM);
  ret.erase(ret.begin());
  return ret;
}
struct SuffixArray {
  int n;
  string s;
  vector<int> sa, rank, lcp;
  vector<vector<int>> t;
  vector<int> lg;
  SuffixArray() {}
  SuffixArray(string _s) {
    n = _s.size();
    s = _s;
    sa = suffix_array(s);
    rank.resize(n);
    for (int i = 0; i < n; i++)
      rank[sa[i]] = i;
    costruct_lcp();
    prec();
    build();
  }
  void costruct_lcp() {
    int k = 0;
    lcp.resize(n - 1, 0);
    for (int i = 0; i < n; i++) {
      if (rank[i] == n - 1) {
        k = 0;
        continue;
      }
      int j = sa[rank[i] + 1];
      while (i + k < n && j + k < n &&
          s[i + k] == s[j + k])  k++;
      lcp[rank[i]] = k;
      if (k)  k--;
    }
  }
  void prec() {
    lg.resize(n, 0);
    for (int i = 2; i < n; i++) lg[i] =
        lg[i / 2] + 1;
  }
  void build() {
    int sz = n - 1;
    t.resize(sz);
    for (int i = 0; i < sz; i++) {
      t[i].resize(LG);
      t[i][0] = lcp[i];
    }
    for (int k = 1; k < LG; ++k) {
      for (int i = 0; i + (1 << k) - 1 <
          sz; ++i) {
        t[i][k] = min(t[i][k - 1], t[i +
            (1 << (k - 1))][k - 1]);
      }
    }
  }
  int query(int l, int r) { // minimum of
      lcp[l], ..., lcp[r]
    int k = lg[r - l + 1];
    return min(t[l][k], t[r - (1 << k) +
        1][k]);
  }
  int get_lcp(int i, int j) { // lcp of
      suffix starting from i and j
    if (i == j) return n - i;
    int l = rank[i], r = rank[j];
    if (l > r) swap(l, r);
    return query(l, r - 1);
  }
  int lower_bound(string &t) {
    int l = 0, r = n - 1, k = t.size(),
        ans = n;
    while (l <= r) {
      int mid = l + r >> 1;
      if (s.substr(sa[mid], min(n -
          sa[mid], k)) >= t) ans = mid,
          r = mid - 1;
      else l = mid + 1;
    }
    return ans;
  }
  int upper_bound(string &t) {
    int l = 0, r = n - 1, k = t.size(),
        ans = n;
    while (l <= r) {
      int mid = l + r >> 1;
      if (s.substr(sa[mid], min(n -
          sa[mid], k)) > t) ans = mid, r
          = mid - 1;
      else l = mid + 1;
    }
    return ans;
  }
  // occurrences of s[p, ..., p + len -
      1]
  pair<int, int> find_occurrence(int p,
      int len) {
    p = rank[p];
    pair<int, int> ans = {p, p};
    int l = 0, r = p - 1;
    while (l <= r) {
      int mid = l + r >> 1;
      if (query(mid, p - 1) >= len)
          ans.first = mid, r = mid - 1;
      else l = mid + 1;
    }
    l = p + 1, r = n - 1;
    while (l <= r) {
      int mid = l + r >> 1;
      if (query(p, mid - 1) >= len)
          ans.second = mid, l = mid + 1;
      else r = mid - 1;
    }
    return ans;
  }
  // count of occ 't' in 's'
  int count_occ(string &t) {
    int lo = 0, hi = n - 1;
    for (int i = 0; i < t.size(); i++) {
      int l = lo, r = hi, ans1 = n + 1;
      while (l <= r) {
        int mid = (l + r) >> 1;
        int idx = sa[mid] + i;
        if (idx < n and s[idx] >= t[i]) {
          ans1 = mid;
          r = mid - 1;
        }
        else l = mid + 1;
      }
      l = lo, r = hi; int ans2 = -1;
      while (l <= r) {
        int mid = (l + r) >> 1;
        int idx = sa[mid] + i;
        if (idx >= n or (idx < n and
            s[idx] <= t[i])) {
          ans2 = mid;
          l = mid + 1;
        }
        else r = mid - 1;
      }
      lo = ans1, hi = ans2;
    }
    return max(0, hi - lo + 1);
  }
};
int32_t main() {
  string s; cin >> s;
  SuffixArray sa(s);
  for (auto x : sa.sa) cout << x << ' ';
      cout << '\n';
}
```

8 Game Theory

8.1 Notes

- First Write a Bruteforce Solution
- If game is not impartial. Greedy, DP may work
- Nim = All XOR
- Misere Nim (Last player who took a stone loses) = Nim + (Corner Case: if all (odd) piles are 1 you lose)
- Bogus Nim = Nim
- Staircase Nim (Given an array where $a[i]$ is the number of stones at index i. Move: Choose any stones (> 1) from index i ($i > 1$) and move them to index $i-1$.) = Odd indexed pile Nim (Even indexed pile doesnt matter, as one player can give bogus moves to drop all even piles to ground)
- **Sprague-Grundy:** Every impartial game under the normal play convention is equivalent to a ONE-HEAP GAME of NIM.
  Grundy value can be calculate by DP/Pattern.



9 Flow

9.1 Flow with L H bound

```
const long long inf = 1LL << 61;
struct Dinic {
  struct edge {int to, rev; long long
      flow, w; int id;};
  int n, s, t, mxid;
  vector<int> d, flow_through;
      vector<int> done;
  vector<vector<edge>>g;
  Dinic() {}
  Dinic(int _n) {
    n = _n + 10; mxid = 0;
    g.resize(n);
  }
  void AddEdge(int u, int v, long long
      w, int id = -1) {
```

```
    edge a = {v, (int)g[v].size(), 0, w,
        id};
    edge b = {u, (int)g[u].size(), 0, 0,
        -1};//for bidirectional edges
        cap(b) = w
    g[u].emplace_back(a);
    g[v].emplace_back(b);
    mxid = max(mxid, id);
  }
  bool bfs() {
    d.assign(n, -1); d[s] = 0;
    queue<int> q; q.push(s);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (auto &e : g[u]) {
        int v = e.to;
        if (d[v] == -1 && e.flow < e.w)
            d[v] = d[u] + 1, q.push(v);
      }
    }
    return d[t] != -1;
  }
  long long dfs(int u, long long flow) {
    if (u == t) return flow;
    for (int &i = done[u]; i <
        (int)g[u].size(); i++) {
      edge &e = g[u][i];
      if (e.w <= e.flow) continue;
      int v = e.to;
      if (d[v] == d[u] + 1) {
        long long nw = dfs(v, min(flow,
            e.w - e.flow));
        if (nw > 0) {
          e.flow += nw;
          g[v][e.rev].flow -= nw;
          return nw;
        }
      }
    }
    return 0;
  }
  long long max_flow(int _s, int _t) {
    s = _s; t = _t;
    long long flow = 0;
    while (bfs()) {
      done.assign(n, 0);
      while (long long nw = dfs(s, inf))
          flow += nw;
    }
    flow_through.assign(mxid + 10, 0);
    for (int i = 0; i < n; i++) for
        (auto e : g[i]) if (e.id >= 0)
        flow_through[e.id] = e.flow;
    return flow;
  }
};
struct LR_Flow {
  Dinic F;
  int n, s, t;
  struct edge {int u, v, l, r, id;};
  vector<edge> edges;
  LR_Flow() {}
  LR_Flow(int _n) {
    n = _n + 10; s = n - 2, t = n - 1;;
    edges.clear();
  }
  void AddEdge(int u, int v, int l, int
      r, int id = -1) {
    assert(0 <= l && l <= r);
    edges.push_back({u, v, l, r, id});
  }
  bool feasible(int _s = -1, int _t =
      -1, int L = -1, int R = -1) {
    if (L != -1) edges.push_back({_t,
        _s, L, R, -1});
    F = Dinic(n);
    long long target = 0;
    for (auto e : edges) {
      int u = e.u, v = e.v, l = e.l, r =
          e.r, id = e.id;
      if (l != 0) {
        F.AddEdge(s, v, l);
        F.AddEdge(u, t, l);
        target += l;
      }
      F.AddEdge(u, v, r - l, id);
    }
    auto ans = F.max_flow(s, t);
    if (L != -1) edges.pop_back();
    if (ans < target) return 0; //not
        feasible
    return 1;
  }
  int max_flow(int _s, int _t) { //-1
      means flow is not feasible
    int mx = 1e9;
    int ans = -1, l = 0, r = mx;
        //l=-mx
    while (l <= r) {
      int mid = l + r >> 1;
      if (feasible(_s, _t, mid, mx)) ans
          = mid, l = mid + 1;
      else r = mid - 1;
    }
    return ans;
  }
  int min_flow(int _s, int _t) { //-1
      means flow is not feasible
    int mx = 1e9;
    int ans = -1, l = 0, r = mx;
        //l=-mx
    while (l <= r) {
      int mid = l + r >> 1;
      if (feasible(_s, _t, 0, mid)) ans
          = mid, r = mid - 1;
      else l = mid + 1;
    }
    return ans;
  }
};
int n, m;  cin >> n >> m;
int dummy_source = 0;
int dummy_sink = n + 1;
Dinic *mr_warlock = new Dinic(n + 2);
int sum = 0;
for (int i = 1; i <= m; i++) {
  int u, v, l, h;    cin >> u >> v >> l
      >> h;
  sum += l; ans[i] = l;
  mr_warlock->AddEdge(u, v, h - l, i);
  mr_warlock->AddEdge(u, dummy_sink, l,
      0);
  mr_warlock->AddEdge(dummy_source, v,
      l, 0);
}
int z =
    mr_warlock->GetMaxFlow(dummy_source,
    dummy_sink);
if (sum != z) {NO; return;}
cal_ans(*mr_warlock);
YES;
for (int i = 1; i <= m; i++) {
  cout << ans[i] << endl;
}
```

9.2 Hopcroft Karp

```
struct HopcroftKarp { //O( E*sqrt(V) )
  static const int MAX = 1005;
  const int INF = 1 << 30;
  vector<int> G[MAX];
  int n, m, match[MAX], dist[MAX];
  // n: number of nodes on left side,
      nodes are numbered 1 to n
  // m: number of nodes on right side,
      nodes are numbered n+1 to n+m
  // G = NIL[0] ? G1[G[1---n]] :
      G2[G[n+1---n+m]]
  void init(int N, int M) {
    memset(match, 0, sizeof match);
    n = N, m = M;
    for (int i = 0; i <= m + n;
        i++)G[i].clear();
  }
  inline void addEdge(int u, int v) {
    G[u].pb(v);
  }
  bool bfs() {
    queue<int> Q;
    for (int i = 1; i <= n; i++) {
      if (match[i] == 0) {
        dist[i] = 0;
        Q.push(i);
      }
      else dist[i] = INF;
    }
    dist[0] = INF;
    while (!Q.empty()) {
      int u = Q.front(); Q.pop();
      if (u != 0) {
        for (int v : G[u]) {
          if (dist[match[v]] == INF) {
            dist[match[v]] = dist[u] + 1;
            Q.push(match[v]);
          }
        }
      }
    }
    return (dist[0] != INF);
  }
  bool dfs(int u) {
    if (u != 0) {
      for (int v : G[u]) {
        if (dist[match[v]] == dist[u] +
            1) {
          if (dfs(match[v])) {
            match[v] = u;
            match[u] = v;
            return true;
          }
        }
      }
      dist[u] = INF;
      return false;
    }
    return true;
  }
  int bpm() {
    int matching = 0;
    while (bfs())
      for (int i = 1; i <= n; i++)
        if (match[i] == 0 && dfs(i))
          matching++;
    return matching;
  }
} HK;
//  HK.init(n, m);
//  HK.addEdge(u, v);
//  int maxMatching = HK.bpm();
// Max independent set = N - matching
// Min node cover = matching
// Min edge cover = N - matching  (if
    egde cover exist)
// Min path cover = N - matching (DAG)
```

9.3 Max-Flow

```
const int INF = 2000000000;
struct Edge {
  int from, to, cap, flow, index;
  Edge(int from, int to, int cap, int
      flow, int index):
    from(from), to(to), cap(cap),
        flow(flow), index(index) {}
};
struct Dinic {
  int N;
  vector<vector<Edge>>G;
  vector<Edge*>dad;
  vector<int>Q;
  Dinic(int N) : N(N), G(N), dad(N),
      Q(N) {}
  void AddEdge(int from, int to, int
      cap) {
    G[from].emplace_back(from, to, cap,
        0, G[to].size());
    if (from == to)
        {G[from].back().index++;}
    G[to].emplace_back(to, from, 0, 0,
        G[from].size() - 1);
  }
  long long BlockingFlow(int s, int t) {
    fill(dad.begin(), dad.end(), (Edge*)
        NULL);
    dad[s] = &G[0][0] - 1;
    int head = 0, tail = 0;
    Q[tail++] = s;
    while (head < tail) {
      int x = Q[head++];
```

```
    for (int i = 0; i < G[x].size();
        i++) {
      Edge &e = G[x][i];
      if ((!dad[e.to]) && ((e.cap -
          e.flow) > 0)) {
        dad[e.to] = &G[x][i];
        Q[tail++] = e.to;
      }
    }
  }
  if (!dad[t]) {return 0;}
  long long totflow = 0;
  for (int i = 0; i < G[t].size();
      i++) {
    Edge *start =
        &G[G[t][i].to][G[t][i].index];
    int amt = INF;
    for (Edge *e = start; amt && (e !=
        dad[s]); e = dad[e->from]) {
      if (!e) {amt = 0; break;}
      amt = min(amt, e->cap - e->flow);
    }
    if (amt == 0) {continue;}
    for (Edge *e = start; amt && (e !=
        dad[s]); e = dad[e->from]) {
      e->flow += amt;
      G[e->to][e->index].flow -= amt;
    }
    totflow += amt;
  }
  return totflow;
}
long long GetMaxFlow(int s, int t) {
  long long totflow = 0;
  while (long long flow =
      BlockingFlow(s, t)) {
    totflow += flow;
  }
  return totflow;
}
bool augment_one(int s, int t) {
  vector<int> parV(N, -1), parE(N, -1);
  queue<int> q;
  q.push(s);
  parV[s] = s;
  while (!q.empty() && parV[t] == -1) {
    int u = q.front(); q.pop();
    for (int i = 0; i < G[u].size();
        i++) {
      Edge &e = G[u][i];
      if ((e.cap - e.flow) > 0 &&
          parV[e.to] == -1) {
        parV[e.to] = u;
        parE[e.to] = i;
        q.push(e.to);
        if (e.to == t) break;
      }
    }
  }
  if (parV[t] == -1) return false;
  int v = t;
  while (v != s) {
    int u = parV[v];
    int ei = parE[v];
    Edge &e = G[u][ei];
    e.flow += 1;
    G[e.to][e.index].flow -= 1;
    v = u;
  }
  return true;
 }
};
void PrintGraph(const Dinic &dinic) {
  for (int u = 0; u < dinic.N; ++u) {
    for (const Edge &e : dinic.G[u]) {
      cout << "From: " << e.from
           << ", To: " << e.to
           << ", Cap: " << e.cap
           << ", Flow: " << e.flow
           << ", ReverseIndex: " <<
               e.index
           << endl;
    }
  }
}
// Initialization : Dinic *mr_warlock=new
    Dinic(n+1);
// Add Edge : mr_warlock->AddEdge(a,b,c);
// Get Max Flow :
    mr_warlock->GetMaxFlow(s,t);
// delete (mr_warlock);
//or
// Initialization : Dinic
    mr_warlock(n+1);
// Add Edge : mr_warlock.AddEdge(a,b,c);
// Get Max Flow :
    mr_warlock.GetMaxFlow(s,t);
// delete: no need
/*
    --augment_one--
    store edge id
    e_id[i][j]=mr_warlock.G[i+1].size();
    u->v
    Edge &e =
        mr_warlock.G[u][e_id[i][j]];
    Edge &rev = mr_warlock.G[u][e.index];
    set :
        e.cap=0;
        rev.cap=0;
    check if possible to switch
    if not
        e.cap = saved_cap;
        e.flow = saved_flow;
        rev.cap = saved_rcap;
        rev.flow = saved_rflow;
*/
```

9.4 Max Flow with Min Cost

```
typedef long long T1;//for cost
typedef long long T2;//for flow
const int maxn = 20100;
const T1 INF = 1e12;
const T2 inf = 1e12;
const T1 eps = 0;
struct Edge {
  int from, to;
  T2 cap, flow;
  T1 cost;
};
struct MCMF {//0-indexed
  int n, m, s, t;
  vector<Edge> edges;
  vector<int> G[maxn];
  int p[maxn], inq[maxn];
  T1 d[maxn];
  T2 a[maxn];
  void init(int n) {
    this->n = n;
    for (int i = 0; i < n; i++)
        {G[i].clear();}
    edges.clear();
  }
  void AddEdge(int from, int to, T2 cap,
      T1 cost) {
    edges.push_back((Edge) {from, to,
        cap, 0, cost});
    edges.push_back((Edge) {to, from, 0,
        0, -cost});
    m = edges.size();
    G[from].push_back(m - 2);
    G[to].push_back(m - 1);
  }
  pair<T1, T2> Mincost(int s, int t) {
      //SPFA
    T1 tot_cost = 0;
    T2 tot_flow = 0;
    while (true) {
      for (int i = 0; i < n; i++) {d[i]
          = INF;}
      memset(inq, 0, sizeof(inq));
      d[s] = 0;
      inq[s] = 1;
      p[s] = 0;
      a[s] = inf;
      queue<int> Q;
      srand(time(NULL));
      Q.push(s);
      while (!Q.empty()) {
        int u = Q.front();
        Q.pop();
        inq[u] = 0;
        for (int i = 0; i < G[u].size();
            i++) {
          Edge& e = edges[G[u][i]];
          if (e.cap > e.flow && d[e.to]
              > d[u] + e.cost + eps) {
            d[e.to] = d[u] + e.cost;
            p[e.to] = G[u][i];
            a[e.to] = min(a[u], e.cap -
                e.flow);
            if (!inq[e.to]) {
              Q.push(e.to);
              inq[e.to] = 1;
            }
          }
        }
      }
      if (abs(d[t] - INF) <= eps) {break;}
      tot_cost += (T1)d[t] * a[t];
      tot_flow += a[t];
      int u = t;
      while (u != s) {
        edges[p[u]].flow += a[t];
        edges[p[u] ^ 1].flow -= a[t];
        u = edges[p[u]].from;
      }
    }
    return {tot_cost, tot_flow};
  }
} mcmf;
```

9.5 Flow Properties

- **Bipartite Matching** = bpm
- **Maximum Independent Set** = n - bpm. Bipartite Graph e min je koyta node remove korle kono edge thakbe na tar man bpm. segula bade baki node gula Independent jar man n - bpm.
- **Vertex Cover** = bpm. Bipartite Graph e min je koyta vertex choose korle sob edge thakbe.
- **Edge Cover** = n - bpm. Bipartite Graph e min je koyta edge choose korle sob vertex kono ekta edge er sathe connected thakbe.

10 Misc.

10.1 [Trick] Median of Medians

Problem: Median of All Sub-Array Medians

Idea: Total Subarray, $k = \frac{n(n+1)}{2}$. We need all subarray medians, sort them and get $\frac{k}{2}+1$. Now, Binary Search on Answer. ok() will return true if $cnt >= \frac{k}{2}+1$, where cnt = count of subarrays whose median is $<= mid$. If true, $r = mid - 1$

10.2 Ternary Search

```
// Problem: Pyramid-ICPC Dhaka 2023
// Given the surface area of a square
    base pyramid, Need to maximize
    volume.
double suface_area;
double fun(double square_area) {
  double base = sqrt(square_area);
  double triangle_area = suface_area -
      square_area;
  double per_triangle_area =
      triangle_area / 4;
  double triangle_height =
      (per_triangle_area * 2) / base;
  double x = base / 2;
  double height = sqrt((triangle_height
      * triangle_height) - (x * x));
  double volume = (base * base * height)
      / 3;
  if (x > triangle_height) volume = 0;
  return volume;
}
int32_t main() {
  cin >> suface_area;
  double l = 0, r = suface_area, ans = -1;
  int it = 100;
  while (it--) {
    double mid1 = l + (r - l) / 3;
    double mid2 = r - (r - l) / 3;
    double x = fun(mid1);
    double y = fun(mid2);
```

```
    if (x > y) {
      ans = x;
      r = mid2;
    }
    else l = mid1;
  }
}
```

10.3 Pigeonhole Principle

- At least 1 subarray of an array of length N must be divisible by N.
- Build all possible sequences of length 10 whose value is between 1 to 100. At least any two sequences will be same.
- For $N >= 100$, at least one 4-tuples XOR is 0.
- For $N > 20$ and $a_i <= 10^6$, at least two subsets has same XOR. Because distinct XOR count can be 10^6 and Total Subset $2^{20} > 10^6$. Means there is a subset whose XOR is 0.

10.4 Matrix Expo

```
using Matrix = array<array<ll, N>, N>;
Matrix matrix_mul(Matrix &A, Matrix &B,
    int M_sz) {
  Matrix res = {};
  for (int i = 0; i < M_sz; i++) {
    for (int j = 0; j < M_sz; j++) {
      for (int k = 0; k < M_sz; k++) {
        res[i][j] = (res[i][j] +
            ((A[i][k] * B[k][j]) % M)) %
            M;
      }
    }
  }
  return res;
}
Matrix matrix_pow(Matrix mat, int n, int
    M_sz) {
  Matrix res = {};
  for (int i = 0; i < M_sz; i++)
      {res[i][i] = 1;}
  while (n) {
    if (n & 1) {res = matrix_mul(res,
        mat, M_sz);}
    if (n > 0) {mat = matrix_mul(mat,
        mat, M_sz);}
    n >>= 1;
  }
  return res;
}
//-----------in main-----------------
Matrix Seq = {{}};
Matrix Pat = {{}}; //need only [N][0]
    size
////build Seq
//for(int i=0; i<(d-1);
    i++){Seq[i+1][i]=1;}
//for(int i=0; i<d;
    i++){Seq[0][i]=a[i]%M;}
//for(int i=0; i<d;
    i++){Pat[d-i-1][0]=f[i]%M;}
Seq = matrix_pow(Seq, n - 1, sz);
Matrix res = matrix_mul(Seq, Pat, sz);
cout << res[0][0] << endl;
```

10.5 2D Prefix Sum

```
const int N = 1e3 + 9;
int a[N][N];
ll pref[N][N];
void query(int x1, int y1, int x2, int
    y2) {
    return pref[x2][y2] - pref[x1 -
        1][y2] - pref[x2][y1 - 1] +
        pref[x1 - 1][y1 - 1]
}
int32_t main() {
  int n, m; cin >> n >> m;
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
      pref[i][j] = pref[i - 1][j] +
          pref[i][j - 1] - pref[i - 1][j
          - 1] + a[i][j];
    }
  }
  int q; cin >> q;
  while (q--) {
    int x1, y1, x2, y2; cin >> x1 >> y1
        >> x2 >> y2;
    cout << query(x1, y1, x2, y2) << '\n';
  }
}
```

10.6 2D Static Range Update

```
    // Add x on a rectangle q times,
    Finally print the array
const int N = 1e3 + 9;
int a[N][N];
ll d[N][N]; // difference array
int32_t main() {
  int n, m; cin >> n >> m;
  int q; cin >> q;
  while (q--) {
    int x1, y1, x2, y2, x; cin >> x1 >>
        y1 >> x2 >> y2 >> x; // add x on
        this rectangle
    d[x1][y1] += x;
    d[x1][y2 + 1] -= x;
    d[x2 + 1][y1] -= x;
    d[x2 + 1][y2 + 1] += x;
  }
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
      d[i][j] += d[i - 1][j] + d[i][j -
          1] - d[i - 1][j - 1];
    }
  }
  // new updated array
  for (int i = 1; i <= n; i++) {
    for (int j = 1; j <= m; j++) {
      cout << d[i][j] + a[i][j] << ' ';
    }
    cout << '\n';
  }
}
```

10.7 CHECK SET CLR

```
bool CHECK(int N, int pos) { return
    (bool)(N & (1ll << pos)); }
void SET(int &N, int pos) { (N |= (1ll
    << pos)); }
void CLR(int &N, int pos) { (N &= ~(1ll
    << pos)); }
```

10.8 Graph Directions

```
int dx[] = {+0, +0, +1, -1, -1, +1, -1,
    +1};
int dy[] = {-1, +1, +0, +0, +1, +1, -1,
    -1};
```

10.9 Custom Hash

```
struct custom_hash {
  static uint64_t splitmix64(uint64_t x) {
    x += 0x9e3779b97f4a7c15;
    x = (x ^ (x >> 30)) *
        0xbf58476d1ce4e5b9;
    x = (x ^ (x >> 27)) *
        0x94d049bb133111eb;
    return x ^ (x >> 31);
  }
  size_t operator()(uint64_t x) const {
    static const uint64_t FIXED_RANDOM =
        chrono::steady_clock::now().time
        _since_epoch().count();
    return splitmix64(x + FIXED_RANDOM);
  }
};
```

10.10 Coordinate Compression

```
int32_t main() {
  vector<int> a({100, 9, 10, 10, 9});
  vector<int> v = a;
  sort(v.begin(), v.end());
  v.resize(unique(v.begin(), v.end()) -
      v.begin());
  for (int i = 0; i < a.size(); i++) {
    a[i] = lower_bound(v.begin(),
        v.end(), a[i]) - v.begin() + 1;
  }
}
```

10.11 Submask Generator

```
int mask = 0; // any value
for (int submask = mask; submask;
    submask = (submask - 1) & mask) {}
```

10.12 Custom Comparators

```
bool cmp(pair<int, int> a, pair<int,
    int> b) { // for vector, arr,..
  if (a.first != b.first) return a.first
      > b.first;
  return a.second < b.second; // must
      return false for equal elements
}
struct cmp { // for set, map, pq,..
  bool operator()(const int& a, const
      int&b) const {
    return a > b;
  }
};
```

10.13 MOD Operations

```
inline ll modAdd(ll a, ll b, ll MOD) {ll
    res = a + b; res += ((res >> 63) &
    MOD); if (res >= MOD) res -= MOD;
    return res;}
inline ll modSub(ll a, ll b, ll MOD) {ll
    res = a - b; res += ((res >> 63) &
    MOD); if (res >= MOD) res -= MOD;
    return res;}
inline ll modMul(ll a, ll b, ll MOD) {a
    %= MOD; b %= MOD; a += ((a >> 63) &
    MOD); b += ((b >> 63) & MOD); return
    (a * b) % MOD; }
inline ll modInverse(ll a, ll MOD)
    {return modPow(a, MOD - 2, MOD);}
inline ll modDiv(ll a, ll b, ll MOD)
    {return modMul(a, modInverse(b,
    MOD), MOD);}
inline ll modMulBigMod(ll a, ll b, ll
    MOD) {a %= MOD; b %= MOD; a += ((a
    >> 63) & MOD); b += ((b >> 63) &
    MOD); return (ll)((__int128)a * b %
    MOD);}
```

10.14 Mex with Array Updates

```
int32_t main() {
  int n, q; cin >> n >> q;
  set<int> missing_numbers;
  for (int i = 0; i <= n + 200; i++) {
    missing_numbers.insert(i);
  }
  int a[n + 1];
  map<int, int> freq;
  for (int i = 1; i <= n; i++) {
    cin >> a[i];
    freq[a[i]]++;
    missing_numbers.erase(a[i]);
  }
  while (q--) {
    int i, x; cin >> i >> x;  // Set
        a[i] = x
    int y = a[i]; a[i] = x;
    missing_numbers.erase(x);
    freq[y]--;
    freq[x]++;
    if (freq[y] == 0) {
      freq.erase(y);
      missing_numbers.insert(y);
    }
    cout << *missing_numbers.begin() <<
        '\n'; // mex after upd
  }
}
```

10.15 Check Eq AlphaQ

```
inline bool better (pair <int, int> P,
    pair<int, int> Q) { // from AlphaQ
   auto [u, x] = P; auto [v, y] = Q;  //
       u * 2^x + v * 2^y <= u + v * 2^(x+y) -- >
       u <= v * 2^y
  long long bound = v;
  for (int i = 0; i < y and bound < inf;
      ++i) bound *= 2;
  return u <= bound;
}
```

11 Geometry

11.1 Convex Hull

```
typedef vector<long long> vll;
typedef long double ld;
#define int long long
struct point {
  int x, y;
  point() {}
  point( int x, int y ) { this->x = x;
      this->y = y; }
  bool operator<( const point &other ) {
    if ( x == other.x ) return y <
        other.y;
    return x < other.x;
  }
};
int crossProduct( point &a, point &b,
    point &c ) {
  return (b.x - a.x) * (c.y - a.y) -
      (b.y - a.y) * (c.x - a.x);
}
vector<point> hull( vector<point> a ) {
  vector<point> v = a;
  sort(v.begin(), v.end());
  vector<point> lower, upper;
  for ( auto& p : v ) {
    while ( lower.size() >= 2 &&
        crossProduct(lower[lower.size()
        - 2], lower.back(), p) < 0 ) //
        <= for coleniar
      lower.pop_back();
    lower.push_back(p);
  }
  for ( int i = v.size() - 1; i >= 0;
      i-- ) {
    point p = v[i];
    while ( upper.size() >= 2 &&
        crossProduct(upper[upper.size()
        - 2], upper.back(), p) < 0 ) //
        <= for coleniar
      upper.pop_back();
    upper.push_back(p);
  }
  lower.pop_back();
  upper.pop_back();
  lower.insert(lower.end(),
      upper.begin(), upper.end());
  return lower;
}
void senritsu() {
  int n; cin >> n;
  vector<point> a(n); for ( int i = 0; i
      < n; i++ ) cin >> a[i].x >> a[i].y;
  a = hull(a);
  n = a.size();
  cout << n << endl;
  for ( auto c : a ) cout << c.x << ' '
      << c.y << endl;
}
```

11.2 Area of 2 polygon intersection

```
typedef vector<long long> vll;
typedef long double ld;
#define int long long
struct point {
  ld x, y;
  point() {}
  point( ld x, ld y ) { this->x = x;
      this->y = y; }
  bool operator<( const point &other ) {
    if ( x == other.x ) return y <
        other.y;
    return x < other.x;
  }
  bool operator==( const point &other ) {
    return ( x == other.x && y ==
        other.y );
  }
  bool operator!=( const point &other ) {
    return ( x != other.x || y !=
        other.y );
  }
  point operator-( const point &other ) {
    return {x - other.x, y - other.y};
  }
};
ld crossProduct( point &a, point &b,
    point &c ) {
  return (b.x - a.x) * (c.y - a.y) -
      (b.y - a.y) * (c.x - a.x);
}
ld crossProduct( point &a, point &b ) {
  return a.x * b.y - a.y * b.x;
}
bool inside( point &a, point &b, point
    &c ) {
  return crossProduct(a, b, c) >= 0;
}
bool intersection( point &a, point &b,
    point &c, point &d, point &ans ) { //
    2 line intersection point
  point ab = b - a;
  point cd = d - c;
  ld det = crossProduct( ab, cd );
  if ( det == 0 ) return false; //
      parallel or collinear
  ld z1 = crossProduct( a, b );
  ld z2 = crossProduct( c, d );
  ans.x = (ld)(z1 * (c.x - d.x) - z2 *
      (a.x - b.x)) / det;
  ans.y = (ld)(z1 * (c.y - d.y) - z2 *
      (a.y - b.y)) / det;
  return true;
}
vector<point> innerhull( vector<point>
    a, vector<point> b ) {
  int n = a.size();
  vector<point> ans = b;
  for ( int i = 0; i < n; i++ ) {
    point x = a[i], y = a[(i + 1) % n];
    vector<point> temp = ans; ans.clear();
    int m = temp.size();
    if ( m == 0 ) break;
    for ( int j = 0; j < m; j++ ) {
      point p = temp[j], q = temp[(j +
          1) % m];
      int pIn = inside(x, y, p);
      int qIn = inside(x, y, q);
      if ( pIn ) ans.push_back(p);
      if ( pIn != qIn ) {
        point cur;
        if ( intersection(x, y, p, q,
            cur) ) {
          ans.push_back(cur);
        }
      }
    }
    if ( !ans.empty() ) {
      vector<point> temp;
      temp.push_back(ans[0]);
      for ( int j = 1; j < ans.size();
          j++ ) {
        if ( ans[j] != ans[j - 1]  ) {
          temp.push_back(ans[j]);
        }
      }
      ans = temp;
    }
  }
  return ans;
}
ld areaOfPolygon( vector<point> a ) {
  if ( a.size() < 3 ) return 0.0;
  a.push_back(a[0]);
  int n = a.size();
  ld ans = 0;
  for ( int i = 0; i < n - 1; i++ ) {
    ans += a[i].x * a[i + 1].y;
    ans -= a[i].y * a[i + 1].x;
  }
  if ( ans < 0 ) ans = -ans;
  return ans / 2.0;
}
void senritsu() {
  int n, m; cin >> n >> m;
  vector<point> a(n), b(m);
  for ( int i = 0; i < n; i++ ) {
    cin >> a[i].x >> a[i].y;
  }
  for ( int i = 0; i < m; i++ ) {
    cin >> b[i].x >> b[i].y;
  }
  vector<point> in = innerhull(a, b);
  cout << fixed << setprecision(4) <<
      areaOfPolygon(in) << endl;
}
```

11.3 Point Segment Distance

```
ld pointSegmentDistance( const point &A,
    const point &B, const point &P ) {
  ld dx = B.x - A.x;
  ld dy = B.y - A.y;
  if ( dx == 0 && dy == 0 )
    return sqrt((P.x - A.x) * (P.x -
        A.x) + (P.y - A.y) * (P.y -
        A.y));
  // projection t of P onto AB in [0,1]
  ld t = ((P.x - A.x) * dx + (P.y - A.y)
      * dy) / (dx * dx + dy * dy);
  if ( t < 0 ) t = 0;
  else if ( t > 1 ) t = 1;
  ld projX = A.x + t * dx;
  ld projY = A.y + t * dy;
  return sqrt((P.x - projX) * (P.x -
      projX) + (P.y - projY) * (P.y -
      projY));
}
```

11.4 Three Point Circle

```
struct circle {
  ld cx, cy, r;
  circle(ld x = 0, ld y = 0, ld radius =
      0) : cx(x), cy(y), r(radius) {}
};
circle circumcircle(const point &A,
    const point &B, const point &C) {
  ld x1 = A.x, y1 = A.y;
  ld x2 = B.x, y2 = B.y;
  ld x3 = C.x, y3 = C.y;
  ld D = 2 * (x1 * (y2 - y3) + x2 * (y3
      - y1) + x3 * (y1 - y2));
  if (D == 0) {
    // Collinear points, no circle
    return circle(0, 0, -1);
  }
  ld Ux = ((x1 * x1 + y1 * y1) * (y2 -
      y3) + (x2 * x2 + y2 * y2) * (y3 -
      y1) + (x3 * x3 + y3 * y3) * (y1 -
      y2)) / D;
  ld Uy = ((x1 * x1 + y1 * y1) * (x3 -
      x2) + (x2 * x2 + y2 * y2) * (x1 -
      x3) + (x3 * x3 + y3 * y3) * (x2 -
      x1)) / D;
  ld R = sqrt((Ux - x1) * (Ux - x1) +
      (Uy - y1) * (Uy - y1));
  return circle(Ux, Uy, R);
}
```

11.5 Circle Line Intersection

```
void circle_line() {
  double r, a, b, c; // ax + by + c = 0
  double cx, cy;  // circle center
  double c1 = c + a * cx + b * cy;
  double x0 = -a * c1 / (a * a + b * b);
  double y0 = -b * c1 / (a * a + b * b);
  if (c1 * c1 > r * r * (a * a + b * b)
      + EPS)
    cout << "no points" << endl;
  else if (abs(c1 * c1 - r * r * (a * a
      + b * b)) < EPS) {
    cout << "1 point" << endl;
    cout << x0 + cx << ' ' << y0 + cy <<
        endl;
  } else {
    double d = r * r - c1 * c1 / (a * a
        + b * b);
    double mult = sqrt(d / (a * a + b *
        b));
    double ax = x0 + b * mult;
    double bx = x0 - b * mult;
    double ay = y0 - a * mult;
    double by = y0 + a * mult;
    cout << "2 points" << endl;
    cout << ax + cx << ' ' << ay + cy <<
        endl;
    cout << bx + cx << ' ' << by + cy <<
        endl;}}
```

Algebraic Formulae:

Square Identities

- $(a+b)^2 = a^2 + 2ab + b^2$
- $(a-b)^2 = a^2 - 2ab + b^2$
- $a^2 - b^2 = (a+b)(a-b)$
- $(x+a)(x+b) = x^2 + (a+b)x + ab$

Three Variables Identities

- $(a+b+c)^2 = a^2+b^2+c^2+2ab+2bc+2ac$
- $a^2+b^2+c^2 = (a+b+c)^2 - 2(ab+bc+ac)$
- $2(ab+bc+ac) = (a+b+c)^2 - (a^2+b^2+c^2)$

Cube Identities

- $(a+b)^3 = a^3+3a^2b+3ab^2+b^3$
- $(a+b)^3 = a^3+b^3+3ab(a+b)$
- $(a-b)^3 = a^3-3a^2b+3ab^2-b^3$
- $(a-b)^3 = a^3-b^3-3ab(a-b)$
- $a^3+b^3 = (a+b)(a^2-ab+b^2)$
- $a^3-b^3 = (a-b)(a^2+ab+b^2)$

Special Three-Variable Cube

- $a^3+b^3+c^3-3abc =$ $(a+b+c)(a^2+b^2+c^2-ab-bc-ca)$
- $a^3+b^3+c^3-3abc =$ $\frac{1}{2}(a+b+c)[(a-b)^2+(b-c)^2+(c-a)^2]$
- $(a-b)^3+(b-c)^3+(c-a)^3 =$ $3(a-b)(b-c)(c-a)$

Corollaries

- If $a+b+c=0$, then $a^3+b^3+c^3=3abc$
- If $a^3+b^3+c^3=3abc$, then $a+b+c=0$ or $a=b=c$
- $a^2+b^2 = (a+b)^2-2ab$
- $a^2+b^2 = (a-b)^2+2ab$
- $(a+b)^2 = (a-b)^2+4ab$
- $(a-b)^2 = (a+b)^2-4ab$
- $a^2+b^2 = \frac{(a+b)^2+(a-b)^2}{2}$
- $ab = \left(\frac{a+b}{2}\right)^2 - \left(\frac{a-b}{2}\right)^2$
- $a^3+b^3 = (a+b)^3-3ab(a+b)$
- $a^3-b^3 = (a-b)^3+3ab(a-b)$

Binomial Expansion

Pascal's Triangle

n	Expansion	Terms
0	1	1
1	$1+y$	2
2	$1+2y+y^2$	3
3	$1+3y+3y^2+y^3$	4
4	$1+4y+6y^2+4y^3+y^4$	5
5	$1+5y+10y^2+10y^3+5y^4+y^5$	6

$$(a+b)^n = \sum_{k=0}^{n} \binom{n}{k} a^{n-k}b^k$$

$$(1+y)^n = \binom{n}{0} + \binom{n}{1}y + \binom{n}{2}y^2 + \cdots + \binom{n}{n}y^n$$

Logarithms

Basic Definitions:

- $\log_a b = x$ if and only if $a^x = b$
- $\log_a(a^x) = x$
- $a^{\log_a b} = b$

Change of Base Formula:

- $\log_a M = \frac{\log_b M}{\log_b a}$
- $\log_a b = \frac{1}{\log_b a}$

Important Theorems:

(i) If $x>0$, $y>0$ and $a \neq 1$ then $x=y$ if and only if $\log_a x = \log_a y$

(ii) If $a>1$ and $x>1$ then $\log_a x > 0$

(iii) If $0<a<1$ and $0<x<1$ then $\log_a x > 0$

(iv) If $a>1$ and $0<x<1$ then $\log_a x < 0$

Angles: Radians and Degrees

Conversions

- Radians to degrees: $\theta^\circ = \theta_{\text{rad}} \times \frac{180}{\pi}$
- Degrees to radians: $\theta_{\text{rad}} = \theta^\circ \times \frac{\pi}{180}$
- Full circle: $360^\circ = 2\pi$ radians
- Half circle: $180^\circ = \pi$ radians
- Right angle: $90^\circ = \frac{\pi}{2}$ radians

Polar Coordinates

- Conversion to Cartesian: $x = r\cos\theta$, $y = r\sin\theta$
- Conversion from Cartesian: $r = \sqrt{x^2+y^2}$, $\theta = \tan^{-1}\left(\frac{y}{x}\right)$
- Distance in polar: $d = \sqrt{r_1^2 + r_2^2 - 2r_1r_2\cos(\theta_2-\theta_1)}$

Rotation by angle θ: $\begin{cases} x' = x\cos\theta - y\sin\theta \\ y' = x\sin\theta + y\cos\theta \end{cases}$

Coordinate Geometry

- **Point:** $P = (x, y)$
- **Distance between two points:** $d = \sqrt{(x_2-x_1)^2+(y_2-y_1)^2}$
- **Manhattan distance:** $d_m = |x_2-x_1| + |y_2-y_1|$
- **Chebyshev distance:** $d_c = \max(|x_2-x_1|, |y_2-y_1|)$

Line Geometry Formulas

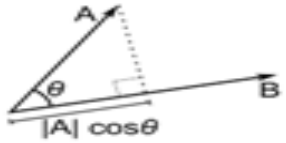
- **Slope:** $m = \frac{y_2-y_1}{x_2-x_1}$
- **Line equation (point-slope):** $y - y_1 = \frac{y_2-y_1}{x_2-x_1}(x-x_1)$
- **General form:** $ax+by+c=0$
- **Distance from point to line:** $\frac{|ax_0+by_0+c|}{\sqrt{a^2+b^2}}$
- **Parallel lines:** $m_1 = m_2$ or $a_1b_2 = a_2b_1$
- **Perpendicular lines:** $m_1 \cdot m_2 = -1$ or $a_1a_2 + b_1b_2 = 0$
- **Distance between parallel lines:** $\frac{|c_2-c_1|}{\sqrt{a^2+b^2}}$
- **Angle between two lines:** $\theta = \tan^{-1}\left|\frac{m_2-m_1}{1+m_1m_2}\right|$
- **Foot of perpendicular:** $\left(\frac{b(bx_0-ay_0)-ac}{a^2+b^2}, \frac{a(-bx_0+ay_0)-bc}{a^2+b^2}\right)$
- **Line through point, parallel to given:** $a(x-x_0)+b(y-y_0)=0$
- **Line through point, perpendicular to given:** $b(x-x_0)-a(y-y_0)=0$
- **Intersection of two lines:** Solve $\begin{cases} a_1x+b_1y+c_1=0 \\ a_2x+b_2y+c_2=0 \end{cases}$
- **Concurrency of three lines:** $\begin{vmatrix} a_1 & b_1 & c_1 \\ a_2 & b_2 & c_2 \\ a_3 & b_3 & c_3 \end{vmatrix} = 0$
- **Section formula (dividing line segment):** $P = \left(\frac{mx_2+nx_1}{m+n}, \frac{my_2+ny_1}{m+n}\right)$
- **Centroid of triangle:** $\left(\frac{x_1+x_2+x_3}{3}, \frac{y_1+y_2+y_3}{3}\right)$
- **Area of triangle using coordinates:** $\frac{1}{2}|x_1(y_2-y_3)+x_2(y_3-y_1)+x_3(y_1-y_2)|$

Vector Operations

Definition: $|\vec{v}| = \sqrt{x^2+y^2}$

Dot Product (Scalar Product) Measures the parallel component of one vector with another. The result is a **scalar**.

- **Algebraic definition:** $\vec{u}\cdot\vec{v} = x_1\cdot x_2 + y_1\cdot y_2$
- **Trigonometric formula:** $\vec{u}\cdot\vec{v} = |\vec{u}|\,|\vec{v}|\cos\theta$
- **Where:** θ is the angle **between** the two vectors
- If $\vec{u}\cdot\vec{v}=0$, the vectors are **perpendicular** ($\cos 90^\circ = 0$)



Cross Product (Vector Product) Measures the perpendicular component and produces a vector **perpendicular** to both.

- **Algebraic definition:** $\vec{u}\times\vec{v} = x_1\cdot y_2 - y_1\cdot x_2$
- **Trigonometric formula:** $|\vec{u}\times\vec{v}| = |\vec{u}|\,|\vec{v}|\sin\theta$
- **Where:** θ is the angle **between** the two vectors
- **Key Insights:**
  - The **magnitude** equals the **area** of the parallelogram formed by $\vec{u}$ and $\vec{v}$
  - If $\vec{u}\times\vec{v}=0$, the vectors are **parallel** ($\sin 0^\circ = 0$)
  - **Sign indicates orientation:**
    - **Positive:** $\vec{v}$ is counter-clockwise from $\vec{u}$
    - **Negative:** $\vec{v}$ is clockwise from $\vec{u}$
  - **CCW (Counter Clockwise Test):** $(x_2-x_1)*(y_3-y_1)-(x_3-y_1)*(y_2-x_1)$
    - >0: Counter-clockwise
    - $=0$: Collinear
    - <0: Clockwise
- **Point on segment check:** $(x_2-x_1)(y_p-y_1)-(x_p-x_1)(y_2-y_1)=0$ and $\min(x_1,x_2) \le x_p \le \max(x_1,x_2)$ and $\min(y_1,y_2) \le y_p \le \max(y_1,y_2)$

Angles and Triangles



For obtuse angle C:

$$AB^2 = AC^2 + BC^2 + 2\cdot BC\cdot CD$$

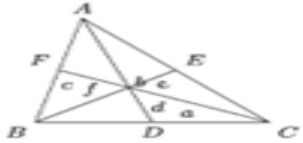


For acute angle c:

$$AB^2 = AC^2 + BC^2 - 2\cdot BC\cdot CD$$

1. If $\angle ACB$ is an obtuse angle, $AB^2 > AC^2 + BC^2$
2. If $\angle ACB$ is a right angle, $AB^2 = AC^2 + BC^2$
3. If $\angle ACB$ is an acute angle, $AB^2 < AC^2 + BC^2$

Theorem of Apollonius The sum of the areas of the squares drawn on any two sides of a triangle is equal to twice the sum of area of the squares drawn on the median of the third side and on either half of that side.



$$d^2 = \frac{2(b^2+c^2)-a^2}{4}$$

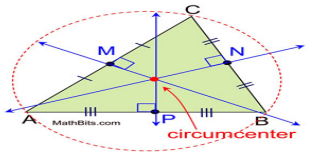
$$e^2 = \frac{2(c^2+a^2)-b^2}{4}$$

$$f^2 = \frac{2(a^2+b^2)-c^2}{4}$$

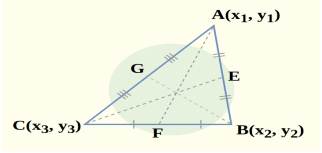
$$\therefore 3(a^2+b^2+c^2) = 4(d^2+e^2+f^2)$$

Triangle Centers

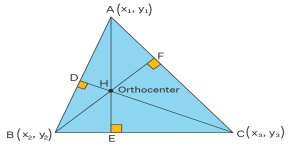
Circumcenter of a Triangle The circumcenter of a triangle is the point of intersection of two perpendicular bisectors of that triangle. Noted that, the perpendicular bisector of the third side of the triangle would pass through the circumcenter too.



Centroid of a Triangle The centroid of a triangle is the point of intersection of three medians of that triangle. The centroid of a triangle divides each median.

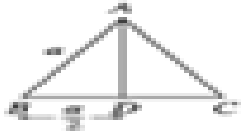


Orthocenter of a Triangle The orthocenter of a triangle is the point of intersection of the perpendiculars drawn from each vertex to their respective opposite side.



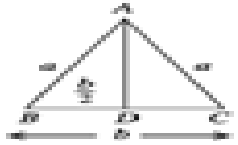
Equilateral triangle:

$$\triangle ABC = \frac{1}{2} \cdot BC \cdot AD = \frac{1}{2} \cdot a \cdot \frac{\sqrt{3}a}{2} = \frac{\sqrt{3}}{4}a^2$$



Isosceles triangle: Area of isosceles $\triangle ABC = \frac{1}{2} \cdot BC \cdot AD$

$$= \frac{1}{2} \cdot b \cdot \frac{\sqrt{4a^2 - b^2}}{2} = \frac{b}{4}\sqrt{4a^2 - b^2}$$



For any triangle, the area given two sides and the included angle is:

$$\text{Area} = \frac{1}{2}ab\sin C$$

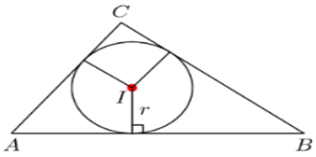
where a and b are two sides and C is the angle between them.

Incircle
The radius of an incircle of a triangle with sides a, b, c and area A is

$$r = \frac{2A}{a+b+c}.$$

The radius of an incircle of a right triangle (the inradius) with legs a, b and hypotenuse c is

$$r = \frac{ab}{a+b+c} = \frac{a+b-c}{2}.$$



Excircle If the circle is tangent to side a of the triangle, the radius is

$$r_a = \frac{K}{s-a},$$

where K is the triangle's area and

$$s = \frac{a+b+c}{2}$$

is the semiperimeter.

Circumradius Let a, b and c denote the triangle's three sides and let A denote the area of the triangle. Then, the measure of the circumradius of the triangle is

$$R = \frac{abc}{4A}.$$

For an equilateral triangle with side length s, the circumradius is

$$R = \frac{s}{\sqrt{3}}.$$

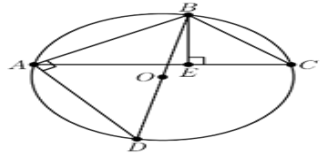
The circumradius can also be expressed in terms of the sides only:

$$R = \frac{abc}{\sqrt{(a+b+c)(-a+b+c)(a-b+c)(a+b-c)}}.$$

By the extended law of sines, we have the relationship:

$$2R = \frac{a}{\sin A},$$

where A is the angle opposite side a.

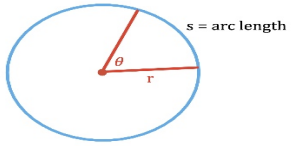


Circle

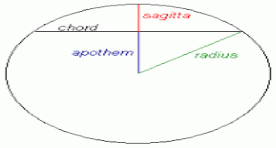
- Circumference: $C = 2\pi r = \pi d$
- Area: $A = \pi r^2 = \frac{\pi d^2}{4}$
- Diameter: $d = 2r$
- Standard form: $(x-h)^2 + (y-k)^2 = r^2$
- General form: $x^2 + y^2 + Dx + Ey + F = 0$ where:
  - Center: $(-\frac{D}{2}, -\frac{E}{2})$
  - Radius: $r = \sqrt{\frac{D^2}{4} + \frac{E^2}{4} - F}$

Circle Parts

- Arc length: $s = r\theta$ (θ in radians) or $s = \frac{\theta}{360} \cdot 2\pi r$
- Sector/Arc area: $A_{\text{sector}} = \frac{1}{2}r^2\theta = \frac{\theta}{360}\pi r^2$



- Chord length: $c = 2r\sin\left(\frac{\theta}{2}\right) = 2\sqrt{r^2 - d^2}$
- Segment area: $A_{\text{segment}} = \frac{1}{2}r^2(\theta - \sin\theta)$
- Sagitta (arrow height): $h = r - \sqrt{r^2 - \left(\frac{c}{2}\right)^2}$



Two Circles Relationships

- Distance between centers: $d = \sqrt{(h_1 - h_2)^2 + (k_1 - k_2)^2}$
- Intersection conditions:
  - Separate: $d > r_1 + r_2$
  - Externally tangent: $d = r_1 + r_2$
  - Intersecting: $|r_1 - r_2| < d < r_1 + r_2$
  - Internally tangent: $d = |r_1 - r_2|$
  - One inside other: $d < |r_1 - r_2|$
- Area of intersection:

$$A = r_1^2\cos^{-1}\left(\frac{d^2 + r_1^2 - r_2^2}{2dr_1}\right) + r_2^2\cos^{-1}\left(\frac{d^2 + r_2^2 - r_1^2}{2dr_2}\right) - \frac{1}{2}\sqrt{S}$$

where
$S = (-d+r_1+r_2)(d+r_1-r_2)(d-r_1+r_2)(d+r_1+r_2)$

Area between three touching circles: The area is given by the general formula for circles of differing radii r_1, r_2, r_3:

$$\text{Area} = \sqrt{r_1r_2r_3(r_1+r_2+r_3)} - \frac{1}{2}\left(r_1^2\theta_1 + r_2^2\theta_2 + r_3^2\theta_3\right)$$

where θ_1, θ_2, and θ_3 are the angles of the triangle formed by their centers, measured in radians. The side lengths of this triangle are $(r_1 + r_2)$, $(r_2 + r_3)$, and $(r_3 + r_1)$. The angles can be found using the Law of Cosines.

Square (side a)

- Perimeter: $P = 4a$
- Area: $A = a^2$
- Diagonal: $d = a\sqrt{2}$

Rectangle (length l, width w)

- Perimeter: $P = 2(l + w)$
- Area: $A = lw$
- Diagonal: $d = \sqrt{l^2 + w^2}$

Parallelogram (base b, height h, sides a, b)

- Perimeter: $P = 2(a + b)$
- Area: $A = bh$

Rhombus (side a, diagonals d_1, d_2)

- Perimeter: $P = 4a$
- Area: $A = \frac{1}{2}d_1d_2$
- Altitude: $h = a\sin\theta$

Trapezoid (bases a, b, height h, legs c, d)

- Perimeter: $P = a + b + c + d$
- Area: $A = \frac{1}{2}(a + b)h$
- Midsegment: $m = \frac{a+b}{2}$

12 Solid Geometry

12.1 Rectangular Solid

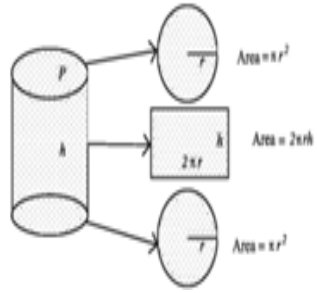
- The diagonal of the rectangular solid $= \sqrt{a^2 + b^2 + c^2}$
- Area of the whole surface: $2(ab + bc + ca)$
- Volume of the rectangular solid = length × width × height $= abc$

12.2 Cube

1. The length of diagonal of the cube
$$= \sqrt{a^2 + a^2 + a^2} = \sqrt{3a^2} = \sqrt{3}a$$
2. The area of the whole surface of the cube
$$= 2(a \cdot a + a \cdot a + a \cdot a) = 2(a^2 + a^2 + a^2) = 6a^2$$
3. The volume of the cube
$$= a \cdot a \cdot a = a^3$$

12.3 Cylinder

1. Area of the base $= \pi r^2$
2. Area of the curved surface = perimeter of the base × height $= 2\pi rh$
3. Area of the whole surface
$$= (\pi r^2 + 2\pi rh + \pi r^2) = 2\pi r(r + h)$$
4. Volume = Area of the base × height $= \pi r^2 h$



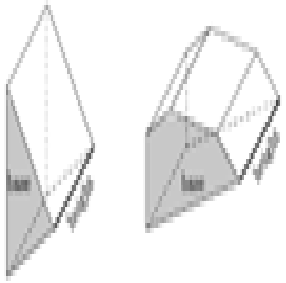
12.4 Prism

1. The area of total surfaces of a prism

$$= 2 \text{ (area of the base)}$$

$$+ \text{ perimeter of the base} \times \text{ height}$$

2. Volume = area of the base × height
3. For a right prism: lateral surface area = perimeter of base × height
4. For a regular prism (with regular polygon base):
   - Base area = $\dfrac{n \times s^2}{4 \times \tan(\frac{\pi}{n})}$ where n is number of sides, s is side length
   - Volume = base area × height



12.5 Pyramid

1. **Total Surface Area**

$$= \text{Area of base} + \text{Area of lateral surfaces}$$

For regular pyramids:

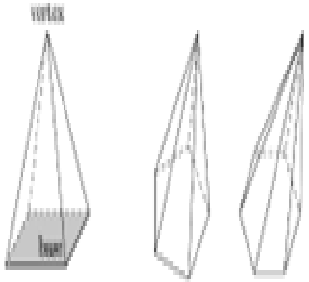
$$= \text{Area of base} + \frac{1}{2}(\text{Perimeter base} \times \text{Slant height})$$

$$\text{Slant height: } l = \sqrt{h^2 + r^2}$$

where h = height, r = inradius of base

2. **Volume**

$$V = \frac{1}{3} \times \text{Area of base} \times \text{Height}$$



12.6 Right Circular Cone

Formulas

- Height: h, Base radius: r, Slant height: l
- Relationship: $l = \sqrt{h^2 + r^2}$

1. **Curved Surface Area: S**

$$= \frac{1}{2} \times \text{circumference} \times \text{slant height}$$

$$= \frac{1}{2} \times 2\pi r \times l = \pi r l$$

2. **Total Surface Area**

$$= \pi r l + \pi r^2 = \pi r(r + l)$$

3. **Volume**

$$V = \frac{1}{3} \times \text{area of base} \times \text{height}$$

$$V = \frac{1}{3}\pi r^2 h$$

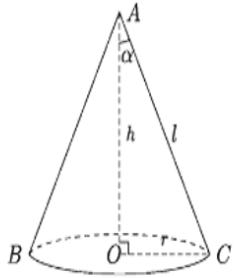
Given a right circular cone with volume V, curved surface area S, base radius r, height h, and semi-vertical angle α:

1. **Curved Surface Area:**

$$S = \frac{\pi h^2 \tan\alpha}{\cos\alpha} = \frac{\pi r^2}{\sin\alpha} \text{ square units}$$

2. **Volume:**

$$V = \frac{1}{3}\pi h^3 \tan^2\alpha = \frac{\pi r^3}{3\tan\alpha} \text{ cubic units}$$



12.7 Sphere

Properties

- Center: O, Radius: $r = OA = OB = OC$
- A plane at distance h from center cuts sphere forming circle with:

$$\text{Radius} = \sqrt{r^2 - h^2}$$

- From right triangle: $OB^2 = OP^2 + PB^2$

Formulas

1. **Surface Area:**

$$A = 4\pi r^2$$

2. **Volume:**

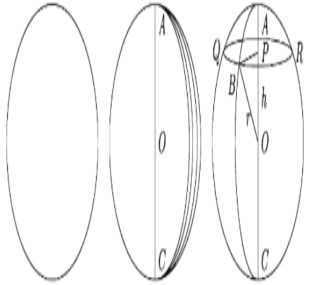
$$V = \frac{4}{3}\pi r^3$$

3. **Section Radius:**

$$R = \sqrt{r^2 - h^2}$$

Related Volumes

- Cone: $V = \frac{1}{3}\pi r^2 h = \frac{1}{3}\pi r^3$
- Hemisphere: $V = \frac{1}{2} \times \frac{4}{3}\pi r^3 = \frac{2}{3}\pi r^3$
- Cylinder: $V = \pi r^2 h = \pi r^3$



Tetrahedron

Regular Tetrahedron

A pyramid with four equilateral triangular faces, six edges, and four vertices.

Formulas (for edge length a)

1. **Surface Area:**

$$A = \sqrt{3} \cdot a^2$$

2. **Volume:**

$$V = \frac{a^3}{6\sqrt{2}}$$

3. **Height:**

$$h = \sqrt{\frac{2}{3}} \cdot a$$

4. **Face Area:**

$$A_{\text{face}} = \frac{\sqrt{3}}{4} \cdot a^2$$

5. **Inradius (radius of inscribed sphere):**

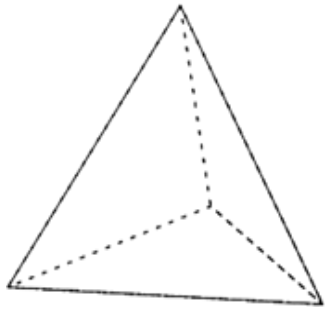
$$r = \frac{a}{\sqrt{24}}$$

6. **Circumradius (radius of circumscribed sphere):**

$$R = \frac{a}{\sqrt{8}}$$

Key Properties

- All faces are congruent equilateral triangles
- All edges have equal length
- 4 vertices, 6 edges, 4 faces (Euler's formula: $V - E + F = 2$)
- Dual polyhedron: Another tetrahedron



Frustum Formulas

Frustum of a Cone

- Base radius: R, Top radius: r, Height: h, Slant height: l
- $l = \sqrt{h^2 + (R - r)^2}$

1. **Curved Surface Area:**

$$A_{\text{curved}} = \pi l(R + r)$$

2. **Total Surface Area:**

$$A_{\text{total}} = \pi[l(R + r) + R^2 + r^2]$$

3. **Volume:**

$$V = \frac{1}{3}\pi h(R^2 + Rr + r^2)$$

Frustum of a Pyramid

- Base area: A_1, Top area: A_2, Height: h
- For square base: side a (base), side b (top)

1. **Lateral Surface Area:**

$$A_{\text{lateral}} = \frac{1}{2}(P_1 + P_2) \times l$$

where P_1, P_2 are perimeters of base and top

2. **Total Surface Area:**

$$A_{\text{total}} = A_{\text{lateral}} + A_1 + A_2$$

3. **Volume:**

$$V = \frac{1}{3}h(A_1 + A_2 + \sqrt{A_1 A_2})$$

4. **For Square Pyramid Frustum:**

$$V = \frac{1}{3}h(a^2 + b^2 + ab)$$

Frustum of a Prism

- Cross-sectional area varies linearly
- Average cross-section method applies

$$V = h \times \frac{A_1 + A_2}{2} \quad \text{(for parallel ends)}$$

Key Relationships

- Similar triangles relate dimensions: $\frac{r}{R} = \frac{h_2}{h_1}$
- For cone frustum: $l^2 = h^2 + (R - r)^2$
- Volume formula derived from difference of two similar cones/pyramids

Angled Frustum (Truncated Cone with Inclined Plane)

General Case: Cone Cut by Inclined Plane

- Base radius: R, Height from base to vertex: H
- Cutting plane inclined at angle θ to base
- Distance from center to cutting plane along axis: h

Cross-Section Properties

1. **Shape of Cut Surface:**
   - Ellipse when cutting plane is inclined
   - Circle only when cutting plane is parallel to base
2. **Ellipse Parameters:**

$$\text{Major axis} = 2R$$

$$\text{Minor axis} = 2R \cdot \cos\theta$$

$$\text{Eccentricity} = \sqrt{1 - \cos^2\theta} = \sin\theta$$

3. **Area of Elliptical Top:**

$$A_{\text{top}} = \pi R^2 \cos\theta$$

Volume Formulas

1. **Using Integration:**

$$V = \int_{h_1}^{h_2} \pi [r(z)]^2 dz$$

where $r(z)$ is the radius at height z

2. **For Right Circular Cone with Inclined Cut:**

$$V = \frac{\pi R^2 H}{3} \left(1 - \left(1 - \frac{h}{H}\right)^3\right) \cdot \frac{1}{\cos\theta}$$

3. **General Formula (Prismoidal Method):**

$$V = \frac{h}{6}\left(A_1 + A_2 + 4A_m\right)$$

where A_1, A_2 are end areas, A_m is mid-section area

Surface Area

1. **Lateral Surface Area:**

$$A_{\text{lateral}} = \pi R\sqrt{R^2 + H^2} - \pi r\sqrt{r^2 + (H - h)^2}$$

2. **Top Surface (Ellipse) Area:**

$$A_{\text{top}} = \pi ab = \pi R^2 \cos\theta$$

Special Cases

- **When $\theta = 0°$:** Standard frustum (parallel bases)
- **When $\theta = 90°$:** Vertical cut through cone
- **When cutting plane passes through vertex:** Forms a triangle cross-section

Key Relationships

- The intersection of a plane and cone always produces a conic section
- Volume depends on both the height of cut and the inclination angle
- For computational purposes, often solved using calculus or numerical methods

Regular Polygon Area General Formula (Side Length = A)

$$A = \frac{n \cdot a^2}{4} \cdot \cot\left(\frac{\pi}{n}\right)$$

Conic Sections Formulas

Ellipse

Standard Forms

- **Horizontal major axis:** $\frac{(x-h)^2}{a^2} + \frac{(y-k)^2}{b^2} = 1$
- **Vertical major axis:** $\frac{(x-h)^2}{b^2} + \frac{(y-k)^2}{a^2} = 1$

The number of diagonals in an n-sided polygon is:

$$\frac{n \cdot (n-3)}{2}$$

If the polygon is regular, you can calculate the measure of each interior angle as:

$$\frac{(n-2) \cdot 180°}{n}$$

Trigonometric Ratio

$$\sec^2\theta - \tan^2\theta = 1$$

$$(\sin\theta)^2 + (\cos\theta)^2 = 1 \quad \text{cosec}^2\theta - \cot^2\theta = 1$$

	0°	30°	45°	60°	90°
sine	0	$\frac{1}{2}$	$\frac{1}{\sqrt{2}}$	$\frac{\sqrt{3}}{2}$	1
cosine	1	$\frac{\sqrt{3}}{2}$	$\frac{1}{\sqrt{2}}$	$\frac{1}{2}$	0
tangent	0	$\frac{1}{\sqrt{3}}$	1	$\sqrt{3}$	udf
cotangent	udf	$\sqrt{3}$	1	$\frac{1}{\sqrt{3}}$	0
secant	1	$\frac{2}{\sqrt{3}}$	$\sqrt{2}$	2	udf
cosecant	udf	2	$\sqrt{2}$	$\frac{2}{\sqrt{3}}$	1

$$\therefore \sin(90° - \theta) = \frac{OM}{OP} = \cos\angle POM = \cos\theta$$

$$\cos(90° - \theta) = \frac{PM}{OP} = \sin\angle POM = \sin\theta$$

$$\tan(90° - \theta) = \frac{OM}{PM} = \cot\angle POM = \cot\theta$$

$$\cot(90° - \theta) = \frac{PM}{OM} = \tan\angle POM = \tan\theta$$

$$\sec(90° - \theta) = \frac{OP}{PM} = \text{cosec}\angle POM = \text{cosec}\theta$$

$$\text{cosec}(90° - \theta) = \frac{OP}{OM} = \sec\angle POM = \sec\theta$$

Value of π = 3.14159265358979323846

Shape	Figure	Surface Area	Volume
Sphere		$SA = 4\pi r^2$ r = radius	$V = \frac{4}{3}\pi r^3$ r = radius
Right Cylinder		$SA = 2\pi rh + 2\pi r^2$ h = height r = radius of base	$V = \pi r^2 h$ h = height r = radius of base
Cone		$SA = \pi rl + \pi r^2$ l = slant height r = radius of base	$V = \frac{1}{3}\pi r^2 h$ h = height r = radius of base
Square Pyramid		$SA = 2sl + s^2$ s = base side length l = slant height	$V = \frac{1}{3}s^2 h$ s = base side length h = height
Rectangular Prism		$SA = 2 \cdot (lw + lh + wh)$ l = length w = width h = height	$V = lwh$ l = length w = width h = height
Cube		$SA = 6s^2$ s = side length (all sides)	$V = s^3$ s = side length (all sides)
General Right Prism		$SA = Ph + 2B$ P = Perimeter of Base h = height (or length) B = area of Base	$V = Bh$ B = area of Base h = height

Shape	Figure	Perimeter	Area
Kite	b, c = sides d_1, d_2 = diagonals	$P = 2b + 2c$	$A = \frac{1}{2}(d_1 d_2)$
Polygon	n = number of sides s = side a = apothem P = perimeter	$P = ns$	$A = \frac{1}{2}a \cdot P$
Ellipse	r_1 = mx rad r_2 = mn rad	$P \approx 2\pi\sqrt{\frac{1}{2}(r_1^2 + r_2^2)}$	$A = \pi r_1 r_2$

```
// Chinese Remainder Theorem
ll educidean_algo(ll a, ll b, ll &x, ll &y) {
  if (b == 0) {x = 1; y = 0; return a;}
  ll x1, y1;
  ll d = educidean_algo(b, a % b, x1, y1);
  x = y1;
  y = x1 - ((a / b) * y1);
  return d;
}

ll invMod_p_q(ll a, ll m) {
  ll x, y;
  educidean_algo(a, m, x, y);
  return (x + m) % m;
}

pair<ll, ll> CRT(ll a1, ll m1, ll a2, ll m2) {
  ll n1, n2, ansM = m1 * m2;
  educidean_algo(m1, m2, n1, n2);
  ll res = (
      ((((a1 * n2) % ansM) * m2) % ansM) +
      ((((a2 * n1) % ansM) * m1) % ansM) +
      ansM
    ) % ansM;
  return {res, ansM};
}
```

```
// Custom Hash for Pairs/Tuples
uint64_t x = ((uint64_t)p.first << 32) ^
    (uint64_t)p.second;
return splitmix64(x + FIXED_RANDOM);
// for tuple
uint64_t x = splitmix64(a + FIXED_RANDOM);
uint64_t y = splitmix64(b + FIXED_RANDOM);
uint64_t z = splitmix64(c + FIXED_RANDOM);
return splitmix64(x ^ (y << 1) ^ (z << 2));
```

```
// Find centroid on a tree
vector<int>centroid;
int centroid_find(int x, int p) {
  int sz = 1;
  bool is_centroid = true;
  for (int u : graph[x]) {
    if (u == p) {continue;}
    int child_sz = centroid_find(u, x);
    if (child_sz > (n / 2)) {is_centroid = false;}
    sz += child_sz;
  }
  if ((n - sz) > (n / 2)) {is_centroid = false;}
  if (is_centroid) {centroid.push_back(x);}
  return sz;
}
```

```
//build
leftSum = new long long[r-l+2];
leftSum[0]=0;
//in build loop
leftSum[cnt] = leftSum[cnt-1];
if(*i <= mid){leftSum[cnt] += *i;}
//in swapAdjacent after 3rd line
leftSum[ind]=leftSum[ind-1]+leftSum[ind+1]-
    leftSum[ind];
//sort the range and return sum of first k indexs
long long kth_smallest_sum(int l, int r, int k){
  if(k==0 || l>r)return 0;
  if(low==high)return 1LL*k*low;
  int lb=pref[l-1];
  int rb=pref[r];
  int cntLeft=rb-lb;
  if(k<=cntLeft){return
      lft->kth_smallest_sum(lb+1,rb,k);}
  long long ans =leftSum[r]-leftSum[l-1];
  return ans +
      rgt->kth_smallest_sum(l-lb,r-rb,k-cntLeft);
}
```